Lecture 3. Context-free grammar

Spring 2026, *Theory of Computation*, IIIS, Tsinghua

Review

- DFAs, NFAs ***recognized*** languages
  - Looks vaguely like an actual computer
  - Reads input, outputs "accept" or "reject"
- REGEXPs ***generated*** languages
  - (0 ∪ 010)* generates strings in the language
  - No explicit computation



Review

- A ***regular expression*** is a string over:
  - symbols in **Σ** (the alphabet) as well as ∅,ε,∪,∘,*,(,)
- *Regular expressions* represent *languages*
  - *a* for some *a* in the alphabet **Σ**
  - ε
  - ∅
  - $(R_1 \cup R_2)$
  - $(R_1 \circ R_2)$
  - (R_1^*)

where R_1, R_2 are regular expressions



Context Free Grammars

- A CFG we've seen already:
- Regular Expressions (not the regular language it represents):

$R \rightarrow A$

$R \rightarrow \varepsilon$

$R \rightarrow \emptyset$

$R \rightarrow (R \circ R)$

$R \rightarrow (R \cup R)$

$R \rightarrow (R)^*$

$A \rightarrow a$

$A \rightarrow b$



Context Free Grammars

- A CFG we've seen already:
- REGEXPS

$R \rightarrow A$

$R \rightarrow \varepsilon$

$R \rightarrow \emptyset$

$R \rightarrow (R \circ R)$

$R \rightarrow (R \cup R)$

$R \rightarrow (R)^*$

$A \rightarrow a$

$A \rightarrow b$

A CFG consists of ***variables***, ***terminals***, and ***production rules***. One variable is the ***start variable***.

To generate a string:

- Make the start variable the initial string
- ***Pick a variable X*** in the string and a ***production rule X → y***
- ***Replace X with y*** and repeat
  (y is a string containing variables ***and*** terminals)

Note! the alphabet of this grammar is {a,b,),(,ε,∅,∘,∪,*}



Production Rules in Action

R	R is the start variable
(R)*	$R \rightarrow (R)^*$
((R∪R))*	$R \rightarrow (R \cup R)$
((A∪R))*	$R \rightarrow A$
((A∪(R∘R)))*	$R \rightarrow (R \circ R)$
((b∪(R∘R)))*	$A \rightarrow b$
((b∪(A∘R)))*	$R \rightarrow A$
((b∪(A∘A)))*	$R \rightarrow A$
((b∪(a∘A)))*	$A \rightarrow a$
((b∪(a∘b)))*	$A \rightarrow b$

stop when there's no more variables



Formal Defn of CFG

- A CFG is a 4-tuple (V,Σ,R,S)
  - V is a set of ***variables***
  - Σ is a set of ***terminals***
  - R is a set of ***rules*** (a variable and a string in $(V \cup \Sigma)^*$)
  - $S \in V$ is the ***start variable***
- Semantics
  - $uAv \Rightarrow uwv$ (uAv ***yields*** uwv) if $A \rightarrow w$ is a rule R
  - $u \Rightarrow^* v$ (u ***derives*** v) if $\exists u_1, u_2, \ldots, u_k$

$$u \Rightarrow u_1 \Rightarrow u_2 \Rightarrow \ldots \Rightarrow u_k \Rightarrow v$$



Language of a CFG

- G = (V,Σ,R,S)

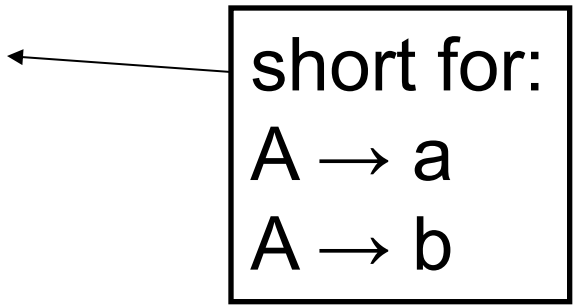
- The language of G is defined as:
  - $L(G) = \{w \in \Sigma^* \mid S \Longrightarrow^* w\}$

    every string of terminals derivable from the start variable S



More compact description

- The REGEXP grammar
  - $R \rightarrow A \mid \varepsilon \mid \emptyset \mid (R \circ R) \mid (R \cup R) \mid (R)^*$
  - $A \rightarrow a \mid b$





What is the language of this CFG?

$S \rightarrow 0S$

$S \rightarrow 1S$

$S \rightarrow \varepsilon$

a) Same as the regular expression 0*1*
b) Same as the regular expression 0* U 1*
c) Same as the regular expression (0 U 1)*
d) None of the above



What is the language of this CFG?

$S \rightarrow T0T$

$T \rightarrow TT$

$T \rightarrow 0T1$

$T \rightarrow 1T0$

$T \rightarrow 0$

$T \rightarrow \varepsilon$

All strings of 0's and 1's that have more 0's than 1's



Are CFG's more powerful than DFA's?

- In other words: is there a CFG G such that the language of G is *not regular*?

a) Yes
b) No



Demonstrating the power of CFG's

- The language $L = \{0^n 1^n \mid n \geq 0\}$ is NOT regular
  - We have shown in the last lecture



Demonstrating the power of CFG's

- The language $L = \{0^n 1^n \mid n \geq 0\}$ is NOT regular.
- Which of these grammars describes the language L?

a) $S \rightarrow AB$
$A \rightarrow 0A \mid \varepsilon$
$B \rightarrow 1B \mid \varepsilon$

b) $S \rightarrow AB$
$A \rightarrow 0A1 \mid \varepsilon$
$B \rightarrow 0B1 \mid \varepsilon$

c) $S \rightarrow 0S1 \mid \varepsilon$

d) $S \rightarrow 00S11 \mid \varepsilon$

e) There is no CFG that describes L



Example

- Design a CFG for the language ABC :
  - ABC = {w | w = $a^i b^j c^k$, i,j,k ≥ 1, and i=j or i=k}
  - e.g., aabbc ∈ ABC, aaabbccc ∈ ABC, but abbccc ∉ ABC



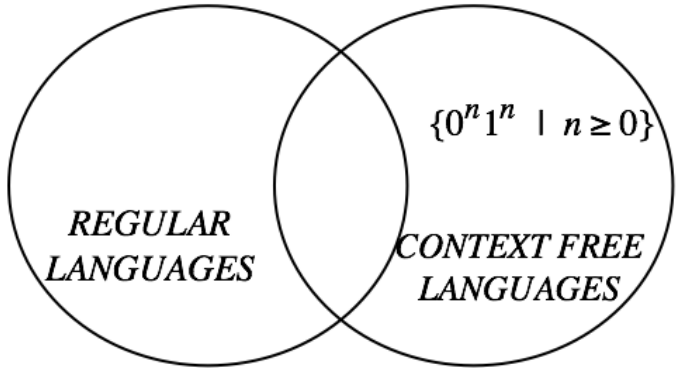
Example

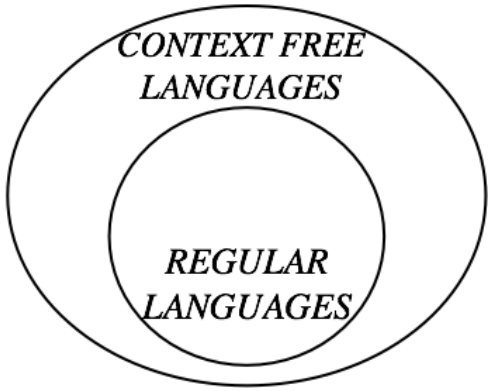
- Design a CFG for the language ABC :
  - ABC = {w | w = $a^i b^j c^k$, i,j,k ≥ 1, and i=j or i=k}
  - e.g., aabbc ∈ ABC, aaabbccc ∈ ABC, but abbccc ∉ ABC

  - S→AC | D
  - A→aAb | ab
  - C→cC | c
  - D→aDc | aBc
  - B→bB| b



- Are CFGs ***strictly better*** than DFAs (or NFAs or REGEXPs) or just different?
  - Two possible scenarios:



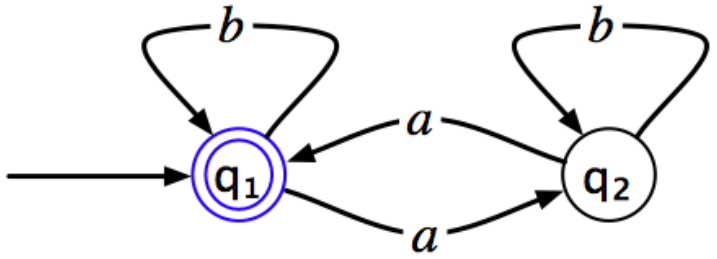




Theorem

- Given a DFA $A = (Q,\Sigma,\delta,q_0,F)$
- Find an equivalent CFG $A' = (V,\Sigma',R,S)$
- Such that L(A) = L(A')

(When A ***"reads"*** a symbol, A' should ***"write"*** it)





Theorem

- Given a DFA $A = (Q,\Sigma,\delta,q_0,F)$
- Find an equivalent CFG $A' = (V,\Sigma',R,S)$
- Such that L(A) = L(A')

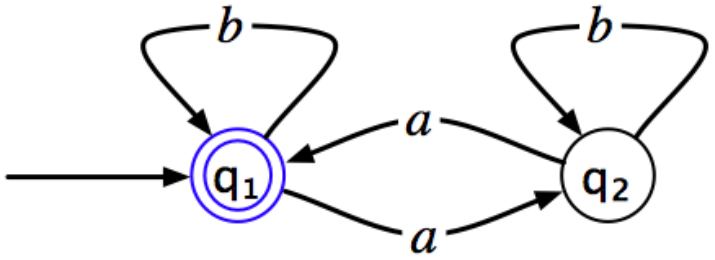
Make a variable R_i in A' for every state q_i in A
Add the rule $R_i \rightarrow aR_j$ to A' if $\delta(q_i, a)=q_j$ in A
R_0 is the start variable if q_0 is the start state
Add the rule $R_i \rightarrow \varepsilon$ if q_i is an accept state in A



Theorem

- Given a DFA $A = (Q,\Sigma,\delta,q_0,F)$
- Find an equivalent CFG $A' = (V,\Sigma',R,S)$
- Such that L(A) = L(A')

Make a variable R_i in A' for every state q_i in A
Add the rule $R_i \rightarrow aR_j$ to A' if $\delta(q_i, a)=q_j$ in A
R_0 is the start variable if q_0 is the start state
Add the rule $R_i \rightarrow \varepsilon$ if q_i is an accept state in A



(Q_1 is the start variable)

$Q_1 \rightarrow bQ_1 \mid aQ_2 \mid \varepsilon$
$Q_2 \rightarrow bQ_2 \mid aQ_1$

Algebraic expression

- We can define the CFG for algebraic expressions:

```
EXPR → a | b | c
EXPR → EXPR + EXPR | EXPR - EXPR
EXPR → EXPR × EXPR | EXPR / EXPR
EXPR → ( EXPR )
```



Compiler

- A CFG for C++ compound statements:

```
<compound stmt> --> { <stmt list> }
  <stmt list> --> <stmt> <stmt list> | epsilon
  <stmt> --> <compound stmt>
  <stmt> --> if ( <expr> ) <stmt>
  <stmt> --> if ( <expr> ) <stmt> else <stmt>
  <stmt> --> while ( <expr> ) <stmt>
  <stmt> --> do <stmt> while ( <expr> ) ;
  <stmt> --> for ( <stmt> ; <expr> ; <stmt> ) <stmt>
  <stmt> --> case <expr> : <stmt>
  <stmt> --> switch ( <expr> ) <stmt>
  <stmt> --> break ; | continue ;
  <stmt> --> return <expr> ; | goto <id> ;
```

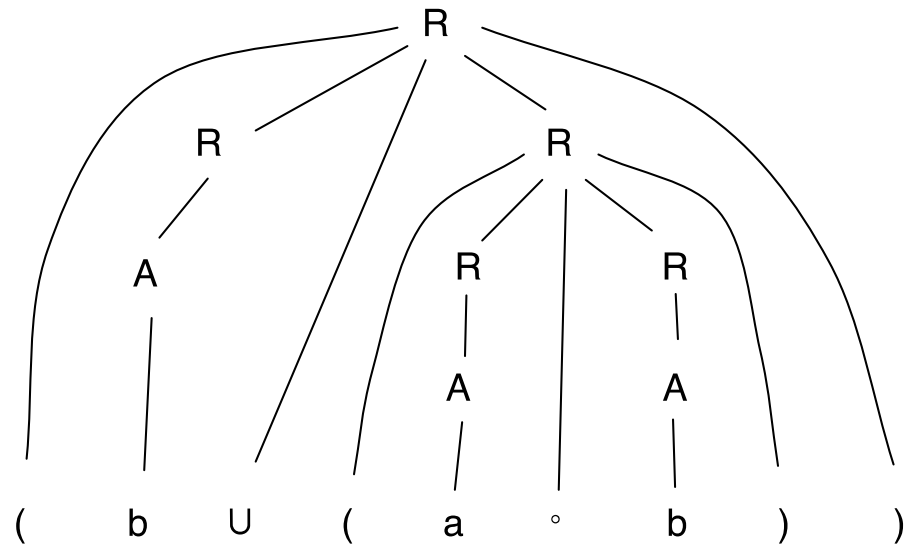


Parse Trees

- Represents the rules used to generate a string
  - Each ***internal node*** labeled with a ***variable***
  - Each ***leaf*** labeled with a ***terminal***
  - The children of a node represent the rule that was applied

$R \rightarrow A \mid \varepsilon \mid \emptyset \mid (R \circ R) \mid (R \cup R) \mid (R)^*$

$A \rightarrow a \mid b$

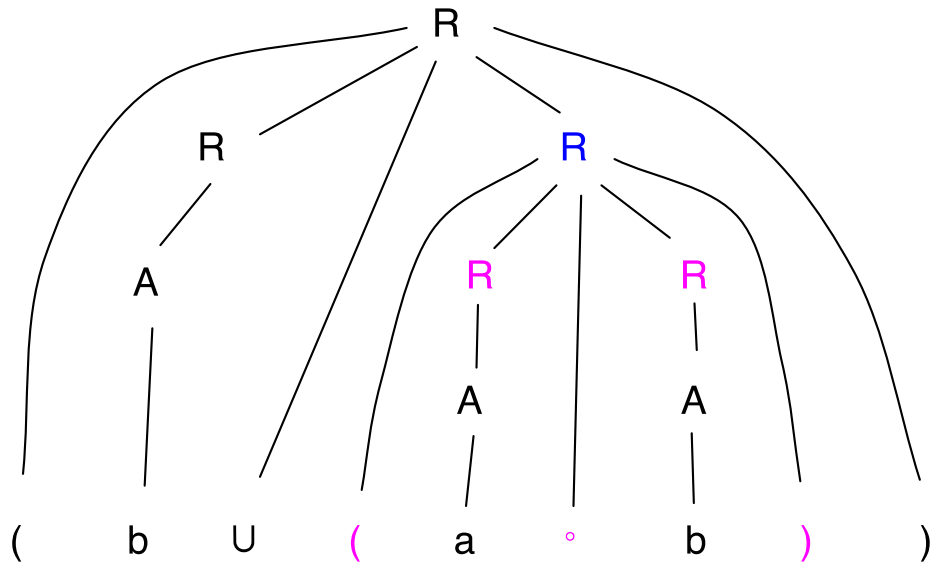




Parse Trees

- Represents the rules used to generate a string
  - Each ***internal node*** labeled with a ***variable***
  - Each ***leaf*** labeled with a ***terminal***
  - The children of a node represent the rule that was applied

R
$\Rightarrow$ (R∪R)
$\Rightarrow$ (A∪R)
$\Rightarrow$ (A∪(R∘R))
$\Rightarrow$ (b∪(R∘R))
$\Rightarrow$ (b∪(A∘R))
$\Rightarrow$ (b∪(A∘A))
$\Rightarrow$ (b∪(a∘A))
$\Rightarrow$ (b∪(a∘b))





Ambiguity

- A grammar is called ***ambiguous*** if a string has ***two distinct parse trees***
  - Not two different derivations

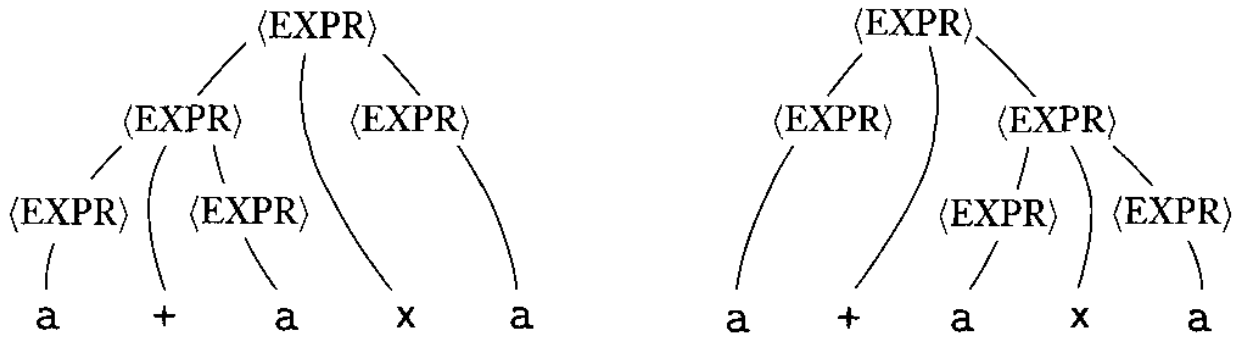


Example

- An ***ambiguous*** grammar:

```
EXPR → a | b | c
EXPR → EXPR + EXPR | EXPR - EXPR
EXPR → EXPR × EXPR | EXPR / EXPR
EXPR → ( EXPR )
```

- The parse trees of "a+a×a":





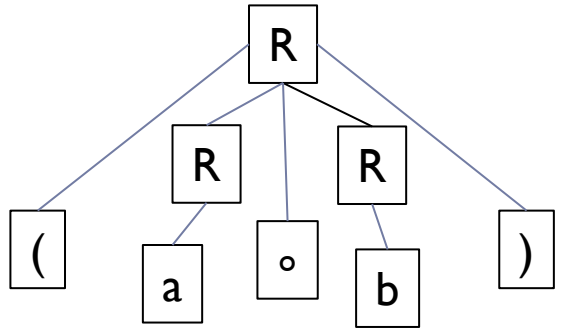
Ambiguity

- A grammar is called ***ambiguous*** if a string has ***two distinct parse trees***
  - Not two different derivations

$R \rightarrow A \mid \varepsilon \mid \emptyset \mid (R \circ R) \mid (R \cup R) \mid (R)^*$

$A \rightarrow a \mid b$

- (a∘b) has different derivations:
  - R⇒ (R∘R)⇒ (A∘R)⇒ (a∘R) ⇒ (a∘A) ⇒ (a∘b)
  - R⇒ (R∘R)⇒ (R∘A)⇒ (R∘b) ⇒ (A∘b) ⇒ (a∘b)
- But they correspond to the same parse tree:

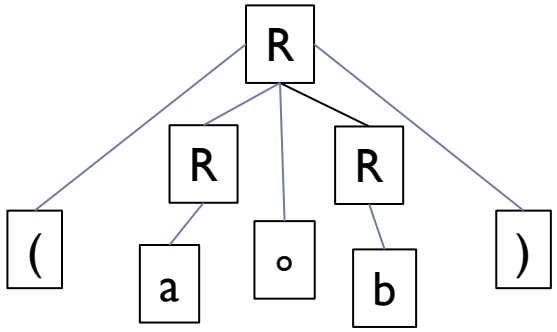




Ambiguity

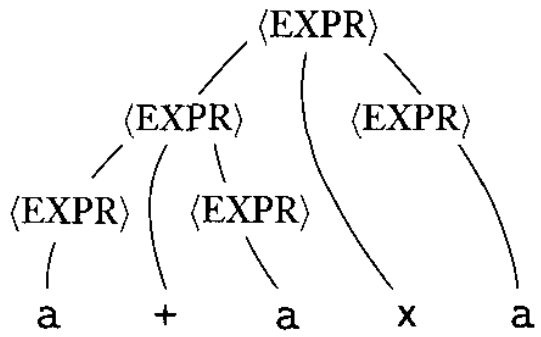
- A grammar is called ***ambiguous*** if a string has ***two distinct parse trees***
  - Not two different derivations

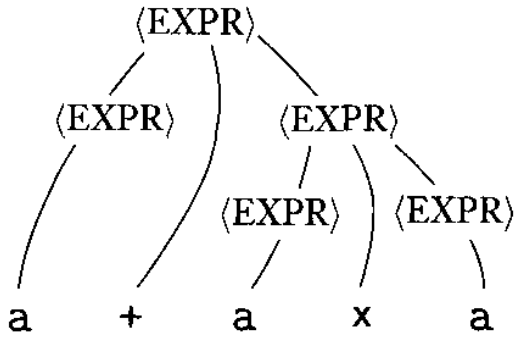
  - Leftmost derivation: at every step the leftmost remaining variable is the one replaced
  - Ambiguous: Two different leftmost derivations
  - R⇒ (R∘R)⇒ (A∘R)⇒ (a∘R) ⇒ (a∘A) ⇒ (a∘b)





The leftmost derivations





EXPR
⟹ EXPR × EXPR
⟹ EXPR + EXPR × EXPR
⟹ a + EXPR × EXPR
⟹ a + a × EXPR
⟹ a + a × a

EXPR
⟹ EXPR + EXPR
⟹ a + EXPR
⟹ a + EXPR × EXPR
⟹ a + a × EXPR
⟹ a + a × a



Unambiguous grammar

- An unambiguous grammar for algebraic expressions?

```
EXPR → a | b | c
EXPR → EXPR + EXPR | EXPR - EXPR
EXPR → EXPR × EXPR | EXPR / EXPR
EXPR → ( EXPR )
```



Unambiguous grammar

- An unambiguous grammar for algebraic expressions:

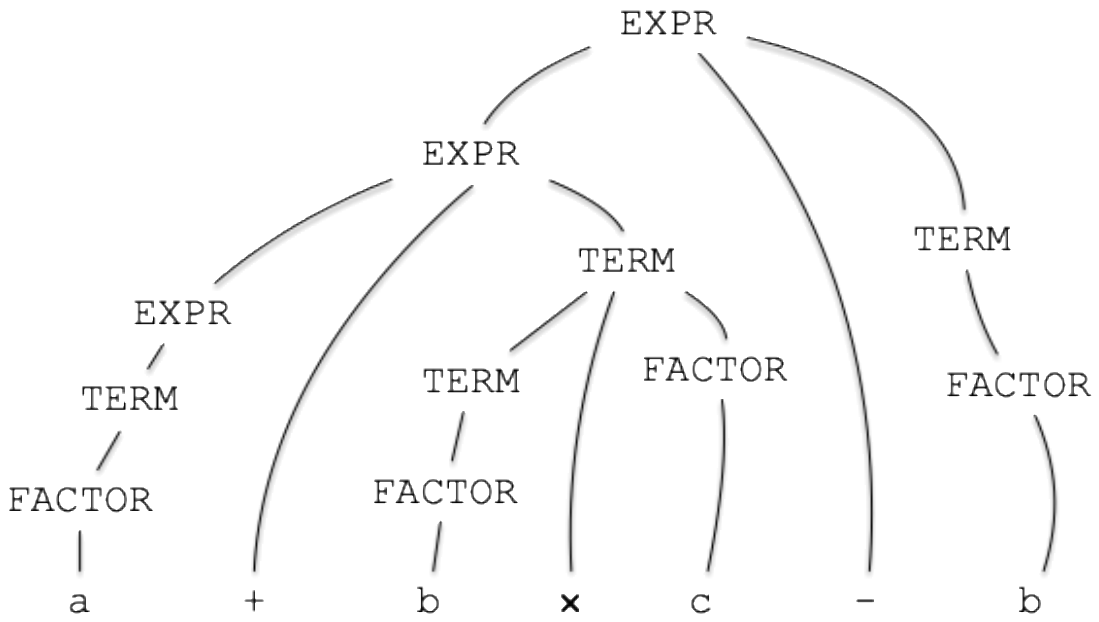
```
EXPR → EXPR + TERM | EXPR - TERM | TERM
TERM → TERM × FACTOR | TERM / FACTOR | FACTOR
FACTOR → ( EXPR ) | a | b | c
```

  - intuitively, we need to designate the order of the computation



EXPR → EXPR + TERM | EXPR − TERM | TERM
TERM → TERM × FACTOR | TERM / FACTOR | FACTOR
FACTOR → (EXPR) | a | b | c

EXPR
⟹ EXPR − TERM
⟹ EXPR + TERM − TERM
⟹ TERM + TERM − TERM
⟹ FACTOR + TERM − TERM
⟹ a + TERM − TERM
⟹ a + TERM × FACTOR − TERM
⟹ a + FACTOR × FACTOR − TERM
⟹ a + b × FACTOR − TERM
⟹ a + b × c − TERM
⟹ a + b × c − FACTOR
⟹ a + b × c − b



Example (Polish notation)

- Another CFG for algebraic expressions:

```
EXPR → a | b | c
EXPR → + EXPR EXPR | - EXPR EXPR
EXPR → × EXPR EXPR | / EXPR EXPR
```

- e.g. a+b×c + a × b c
  (a+b)×c × + a b c

- Is this CFG ambiguous?



```
EXPR → a | b | c
EXPR → + EXPR EXPR | - EXPR EXPR
EXPR → × EXPR EXPR | / EXPR EXPR
```

- Every time we pick the rightmost operator:

```
× + a × b c + - c a b        (a+b×c)×(c-a+b)
× + a × b c + EXPR b
× + a × b c EXPR
× + a × b c EXPR
× + a EXPR EXPR
× EXPR EXPR
EXPR
```



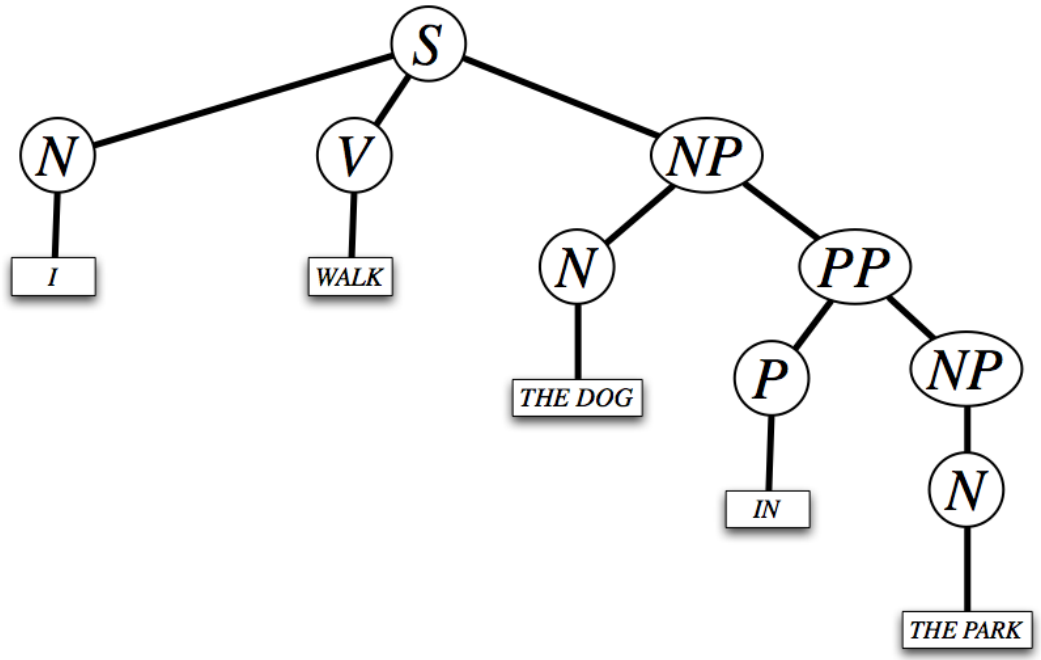
Why they're called "***grammars***"

- **<S>** → **<N><V><NP>**
- **<V>** → eat | walk | talk
- **<N>** → I | you | he | she | it | the dog | the guy | the park
- **<NP>** → **<N>** | **<N><PP>**
- **<PP>** → **<P><NP>**
- **<P>** → in | on | at

<S>
<N><V><NP>
I <V><NP>
I walk <NP>
I walk <N><PP>
I walk the dog <PP>
I walk the dog <P><NP>
I walk the dog in <NP>
I walk the dog in the park
stop when there's no more variables

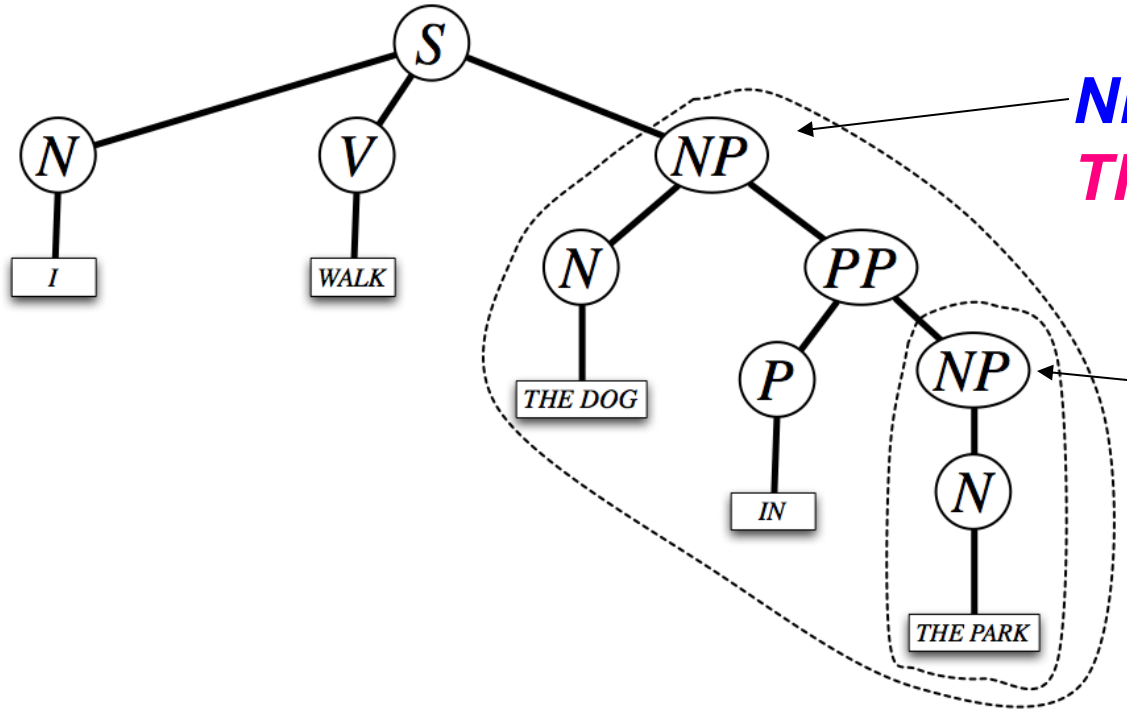


Surgery on Parse Trees





Surgery on Parse Trees



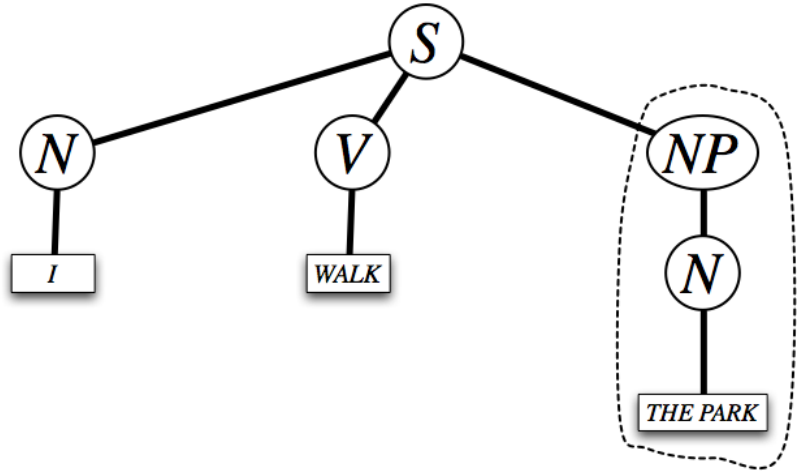
NP generates
THE DOG IN THE PARK

NP also generates
THE PARK

Can replace one subtree with the other
and still have a grammatical sentence



Surgery on Parse Trees

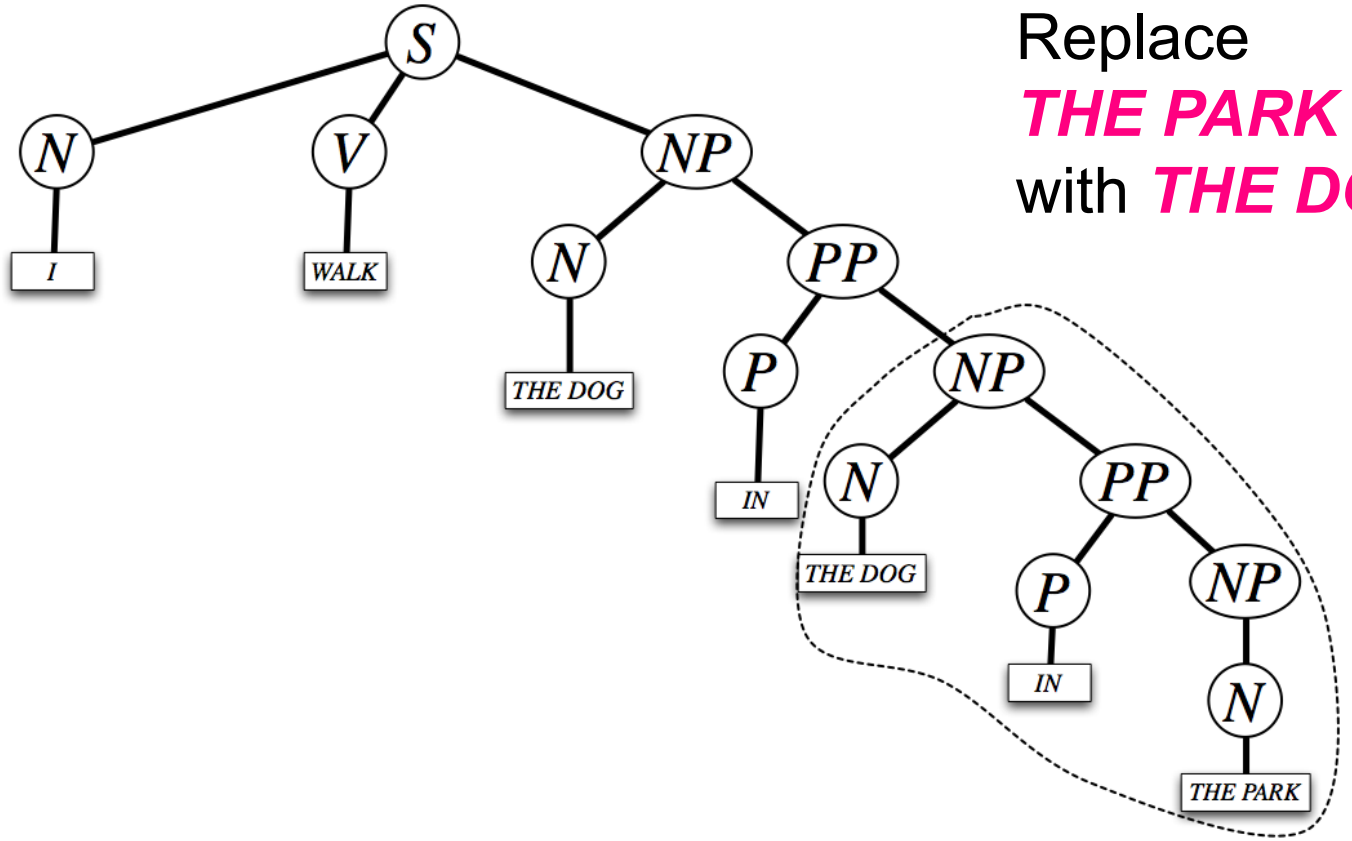


Replace
THE DOG IN THE PARK
with ***THE PARK***

Can replace one subtree with the other
and still have a grammatical sentence



Surgery on Parse Trees



Replace
THE PARK
with ***THE DOG IN THE PARK***

Can replace one subtree with the other
and still have a grammatical sentence



Example Problem

S → NP VP

NP → CN | CN PP

VP → CV | CV PP

PP → P CN

CN → A N

CV → V | V NP

A → a | the

N → boy | girl | flower

V → touches | likes | sees

P → with

S = Sentence

NP = Noun-Phrase

VP = Verb-Phrase

PP = Prep-Phrase

CN = Complex-Noun

CV = Complex-Verb

A = Article

N = Noun

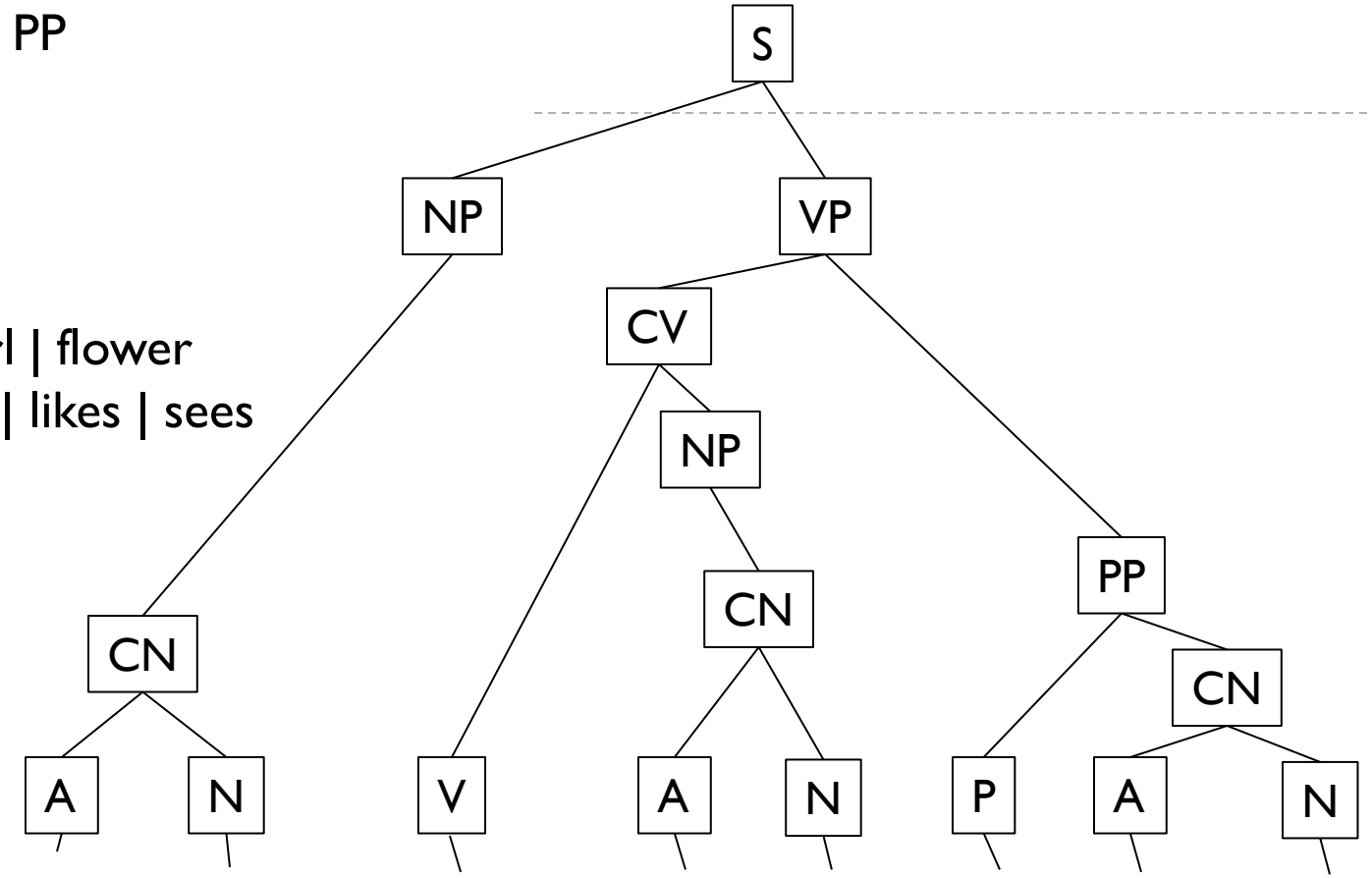
V = Verb

P = Preposition



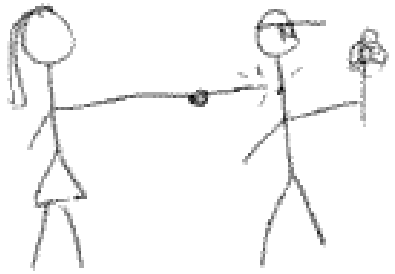
Draw Parse Tree

S -> NP VP
NP -> CN | CN PP
VP -> CV | CV PP
PP -> P CN
CN -> A N
CV -> V | V NP
A -> a | the
N -> boy | girl | flower
V -> touches | likes | sees
P -> with



- "The girl touches the boy with the flower."

(a)

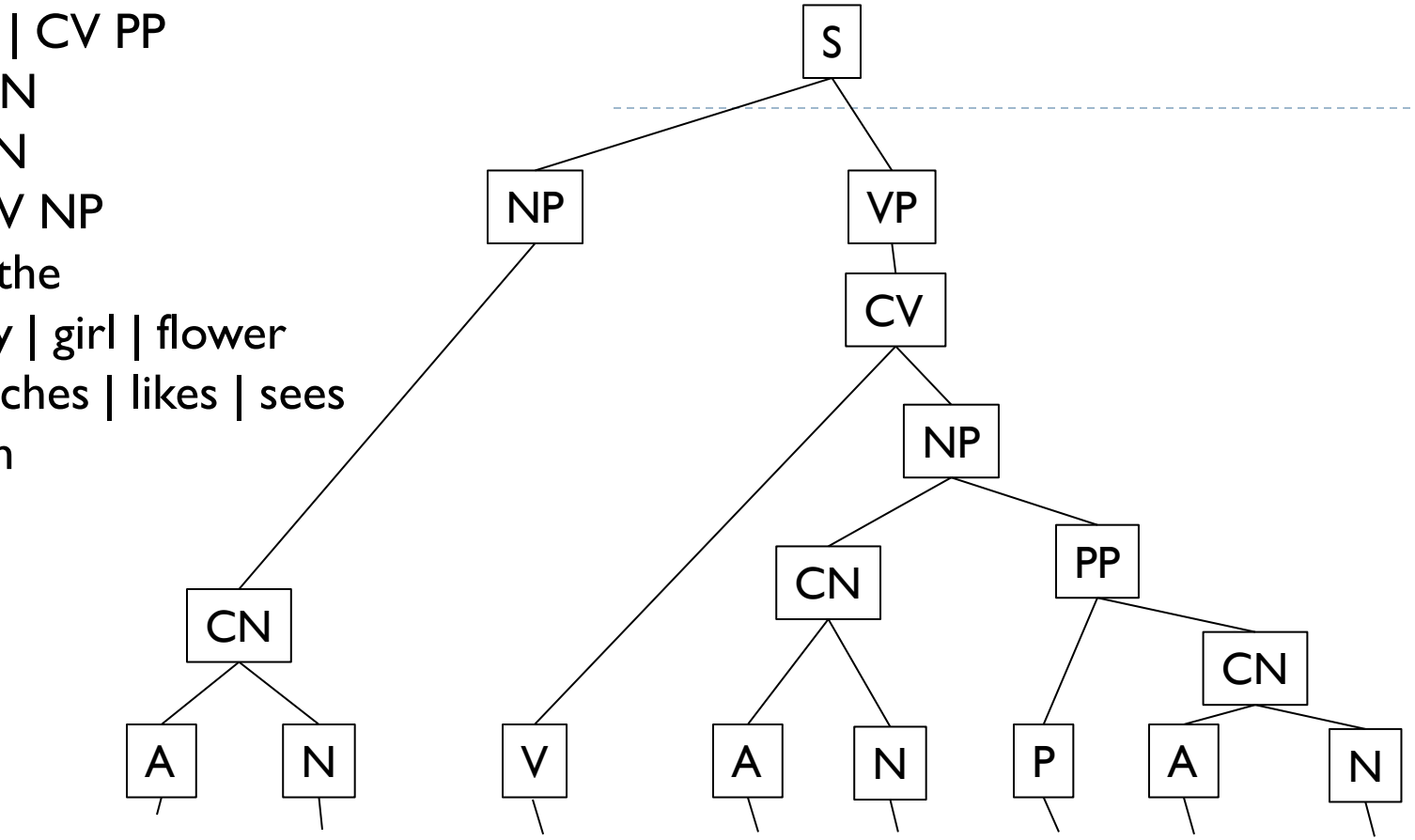


(b)



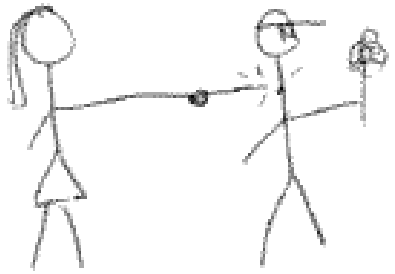
Draw Parse Tree

S -> NP VP
NP -> CN | CN PP
VP -> CV | CV PP
PP -> P CN
CN -> A N
CV -> V | V NP
A -> a | the
N -> boy | girl | flower
V -> touches | likes | sees
P -> with



- "The girl touches the boy with the flower."

(a)



(b)



Ambiguity in Human Languages

- Context-free grammar was developed in mid-1950s by linguist *Noam Chomsky*, in order to formulate the grammar of human languages

- But human languages are definitely not "context-free"



- Is there a method to remove ambiguity from any CFG?

- In fact, even checking whether a CFG is ambiguous is undecidable!
  - no Turing machine can do that



Ambiguity

- A grammar is called ***ambiguous*** if a string has ***two distinct parse trees***

- A CFL is called ***inherently ambiguous*** if every CFG for the language is ambiguous



- Is this grammar ambiguous? Is the language inherently ambiguous?

$$S \rightarrow AS \mid \varepsilon$$

$$A \rightarrow A1 \mid 0A1 \mid 01$$



$S \rightarrow AS \mid \varepsilon$

$A \rightarrow A1 \mid 0A1 \mid 01$

- Is this grammar ambiguous?
  - Yes
  - consider 00111



S → AS | ε
A → A1 | 0A1 | 01

- Is the language inherently ambiguous?
  - No

S → AS | ε
A → 0A1 | B
B → B1 | 01

generates the same language and is not ambiguous



An example of inherently ambiguous CFL

- The example we have seen:

$$L = \{w \mid w = a^i b^j c^k, \quad i,j,k \geq 1, \quad \text{and} \quad i=j \text{ or } i=k\}$$

A Direct Proof of the Inherent Ambiguity of a Simple Context-Free Language, Hermann Maurer, *Journal of ACM*, 1969



Closure Properties

- Good Exercise: Prove that CFLs are closed under
  - Union
  - Concatenation
  - Kleene Star

- We will see that CFLs are NOT closed under intersection. (Is this strange?)
- But CFLs are closed under intersection with REGULAR languages.



Why called “context-free”

- Any variable can be replaced by all rules regardless of the “context”

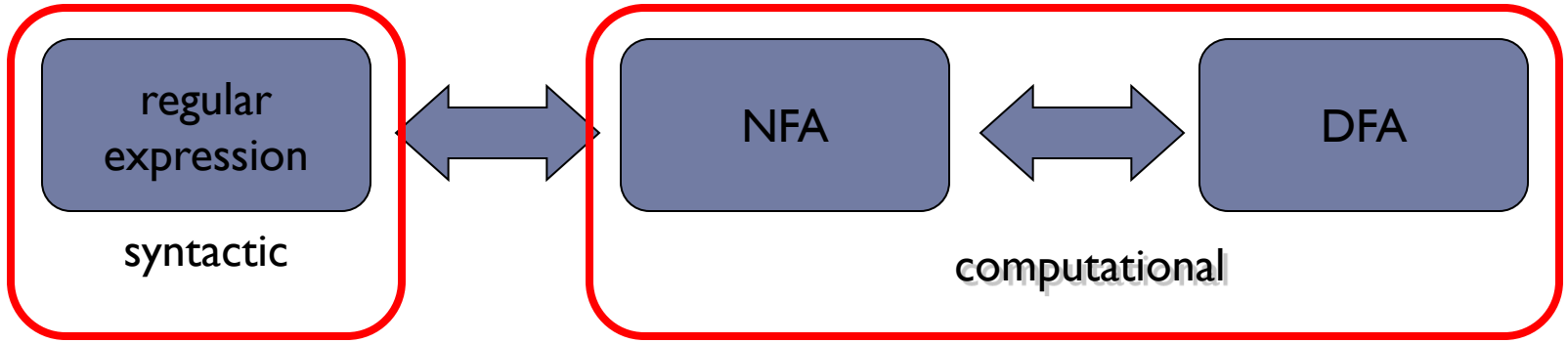
- Context-sensitive grammar:
  - $\alpha A \beta \rightarrow \alpha \gamma \beta$

  - Difficulty: checking whether a string belongs to a CSG G is PSPACE-complete
  - Checking whether a CSG is empty is undecidable, while it is decidable for a CFG

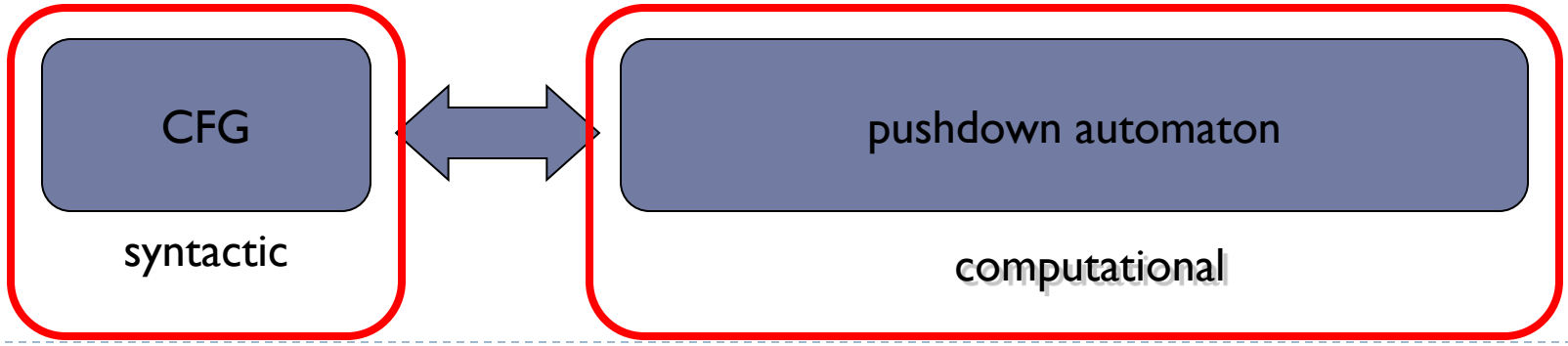


Pushdown Automata-Motivation

- We had two ways to describe regular languages



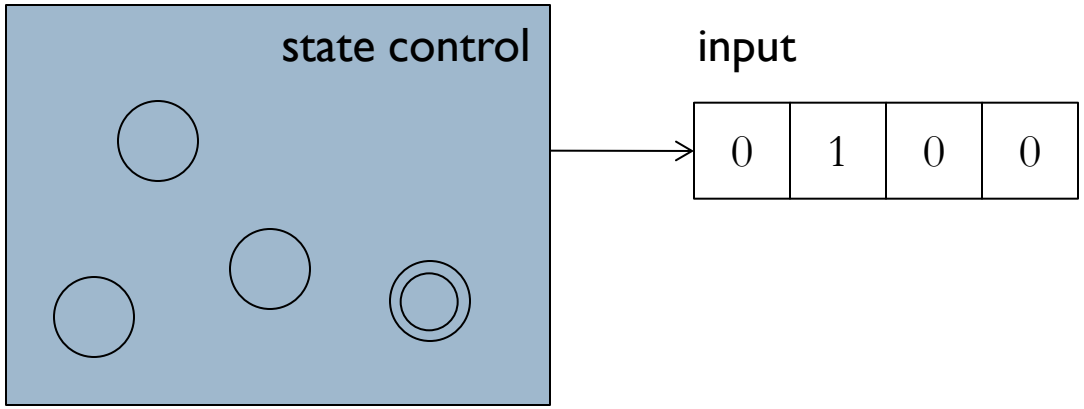
- How about context-free languages?





Pushdown automata versus NFA

- Since context-free is more powerful than regular, pushdown automata must generalize NFAs

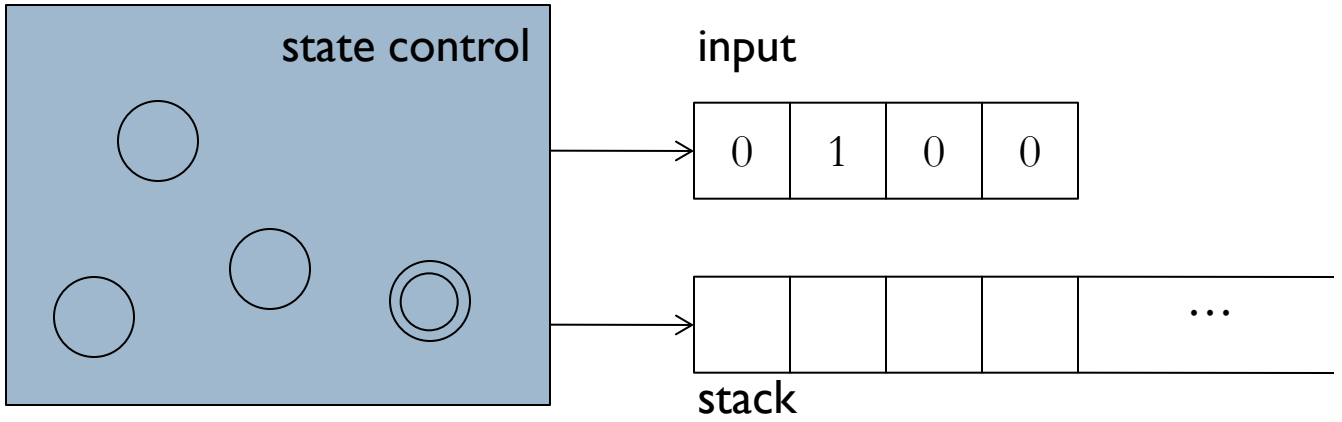


NFA



Pushdown automata

- A pushdown automaton has access to a stack, which is a potentially infinite supply of memory

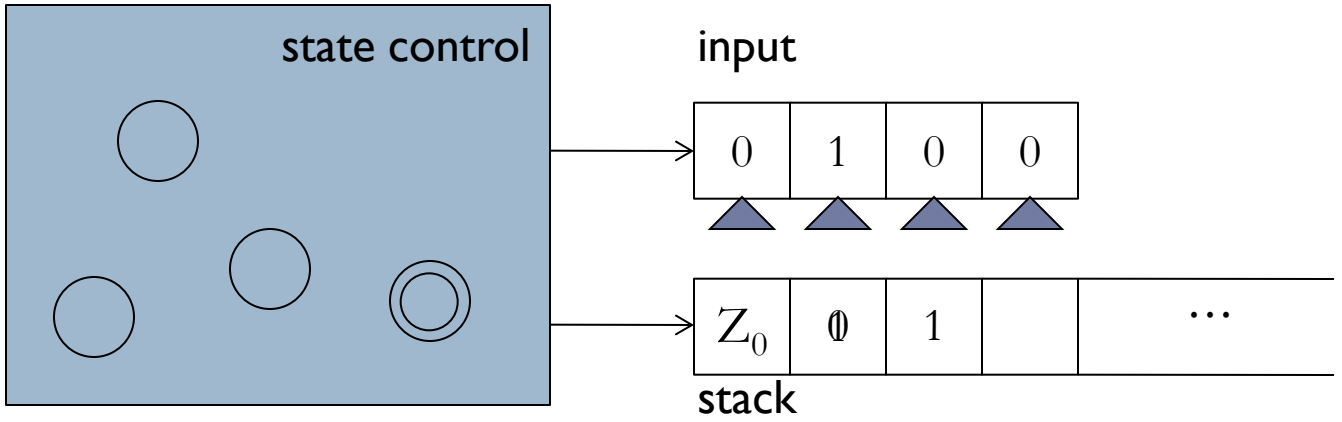


pushdown automaton (PDA)



Pushdown automata

- As the PDA is reading the input, it can push / pop symbols in / out of the stack



pushdown automaton (PDA)



Rules for pushdown automata

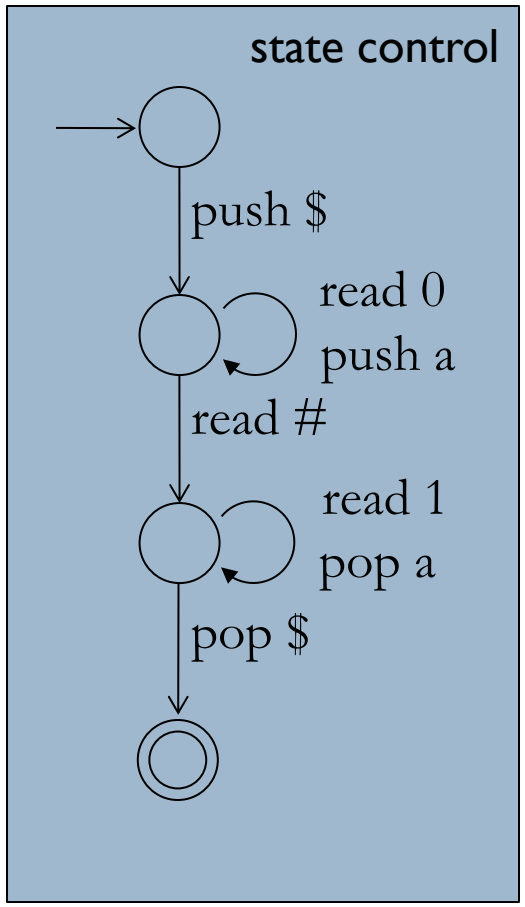
- Multiple and ε-transitions are allowed (nondeterministic!)
- Stack is always accessed from the top
- Each transition can pop and/or push into stack
- Transitions depend on input symbol and on last symbol popped from stack

- Automaton accepts reaching an accepting state

  (another definition: automaton accepts when the stack becomes empty)



Example



$$L = \{0^n\#1^n : n \geq 0\}$$

input

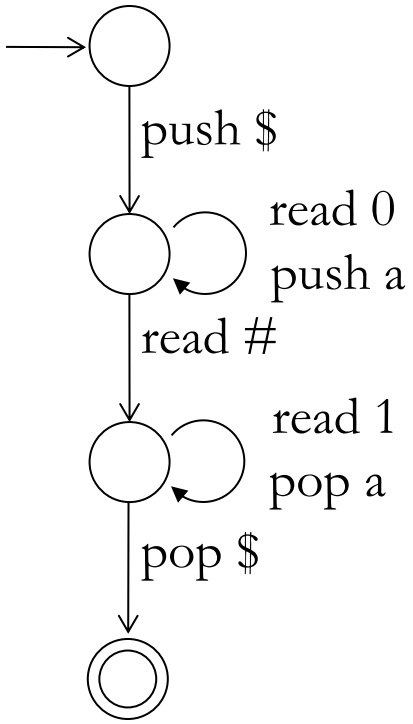
0	0	0	#	1	1	1

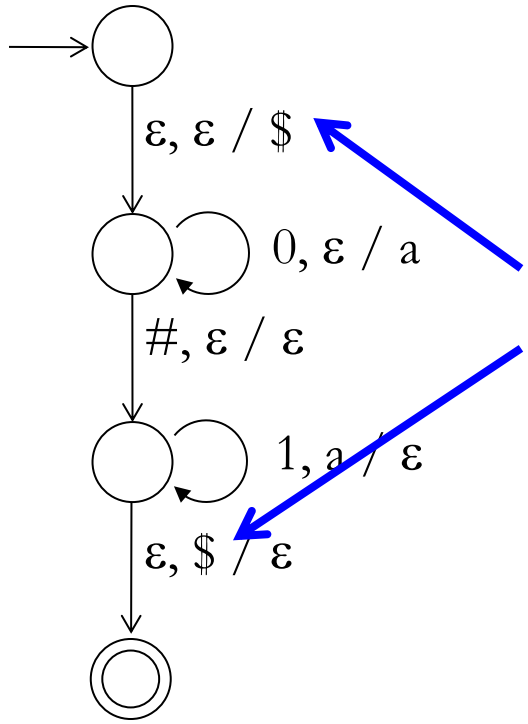
$	a	a	a	…

stack



Shorthand notation





No explicit mechanism
to test for an empty stack
Convention:
we use $ for "end of stack"

read, pop / push



Formal definition

A pushdown automaton is a 6-tuple $(Q, \mathbf{\Sigma}, \mathbf{\Gamma}, \delta, q_0, F)$:

- Q is a finite set of states;
- $\mathbf{\Sigma}$ is the input alphabet;
- $\mathbf{\Gamma}$ is the stack alphabet
- q_0 in Q is the initial state;
- $F \subseteq Q$ is a set of final states;
- δ is the transition function

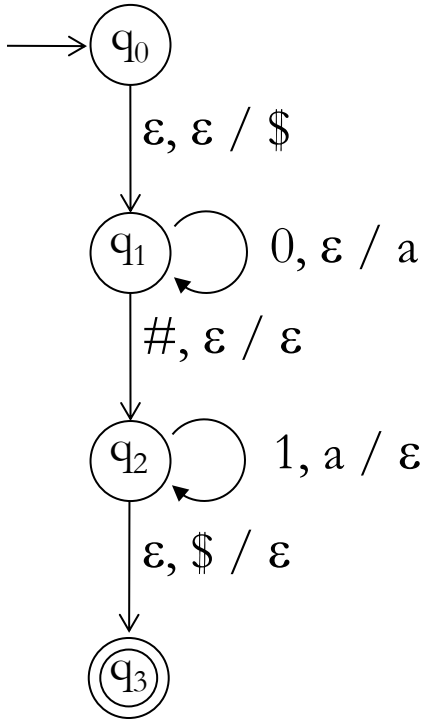
$$\delta: Q \times (\mathbf{\Sigma} \cup \{\varepsilon\}) \times (\mathbf{\Gamma} \cup \{\varepsilon\}) \rightarrow 2^{Q \times (\Gamma \cup \{\varepsilon\})}$$

state input symbol pop symbol state push symbol



Example

Note: PDA cannot read top of stack without popping



$\Sigma = \{0, 1, \#\}$

$\Gamma = \{\$, a\}$

$\delta(q_1, 0, \varepsilon) = \{(q_1, a)\}$

$\delta(q_1, 1, \varepsilon) = \emptyset$

$\delta(q_1, \#, \varepsilon) = \{(q_2, \varepsilon)\}$

$\delta(q_1, 0, a) = \emptyset$

...

δ: $Q \times (\mathbf{\Sigma} \cup \{\varepsilon\}) \times (\mathbf{\Gamma} \cup \{\varepsilon\}) \rightarrow$ subsets of $Q \times (\Gamma \cup \{\varepsilon\})$



Formal Defn. of "Acceptance"

- PDA $(Q, \Sigma, \Gamma, \delta, q_0, F)$ accepts string $w \in \Sigma^*$ if:
  - w can be written $x_1 x_2 \ldots x_m$ each $x_i \in \Sigma \cup \{\varepsilon\}$
    (e.g., 'abba' can be written 'aεbεεba')
  - $\exists\ r_0, r_1, \ldots, r_m \in Q$ *a sequence of m+1 states*
  - $\exists\ s_0, s_1, \ldots, s_m \in \Gamma^*$ *stack contents*
- *such that*
  - $r_0 = q_0$, $s_0 = \varepsilon$
  - $(r_i, b) \in \delta(r_{i-1}, x_i, a)$, where $s_{i-1} = at$ and $s_i = bt$ for $1 \le i \le m$
  - $r_m \in F$



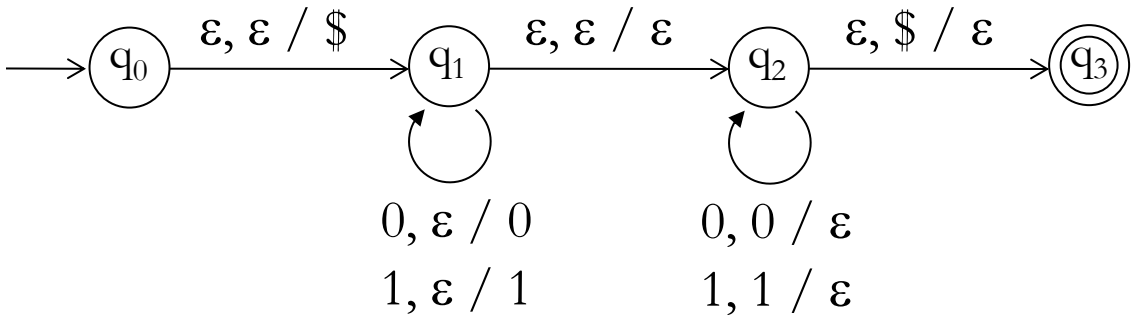
Example 1

$L = \{ww^R : w \in \Sigma^*\}$

$\Sigma = \{0, 1\}$

$\varepsilon, 00, 0110 \in L$

$1, 011, 010 \notin L$



Example 2

$L = \{w: w = w^R, w \in \Sigma^*\}$

$\Sigma = \{0, 1\}$

$\varepsilon, 1, 00, 010, 0110 \in L$

$011 \notin L$



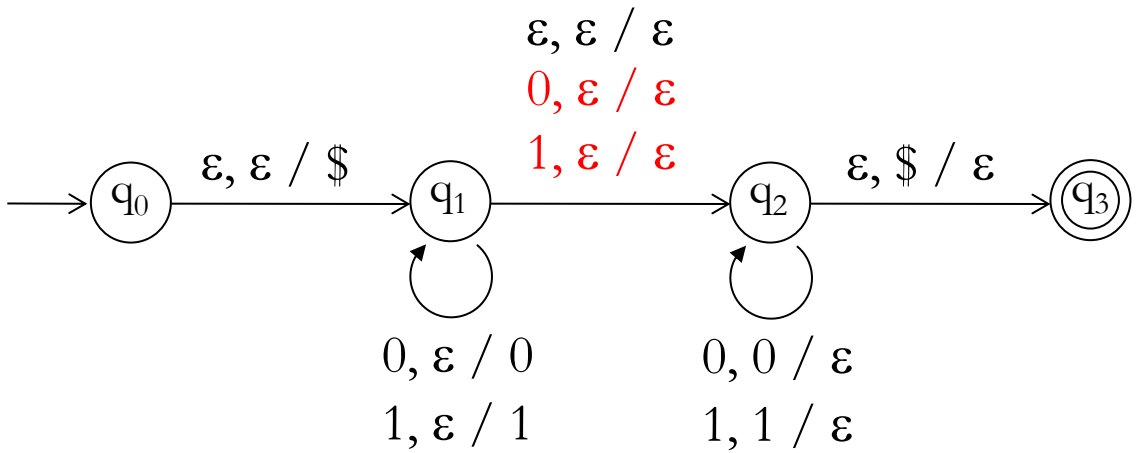
Example 2

$L = \{w: w = w^R, w \in \Sigma^*\}$

$\Sigma = \{0, 1\}$

$\varepsilon, 1, 00, 010, 0110 \in L$

$011 \notin L$



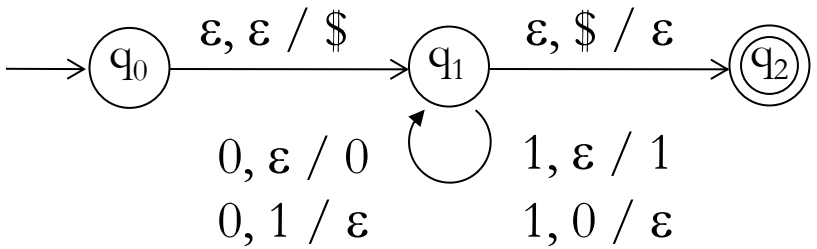


Example 3

$L = \{w: w \text{ has same number of 0s and 1s}\}$ $\Sigma = \{0, 1\}$

Strategy: Stack keeps track of excess of 0s or 1s

If at the end, stack is empty, number is equal





Example 4

$L = \{w: w \text{ has two 0-blocks with same number of 0s}\}$

01011, 001011001, 10010101001

allowed

01001000, 01111

not allowed

Strategy: Guess where first block starts

Record 0s on stack

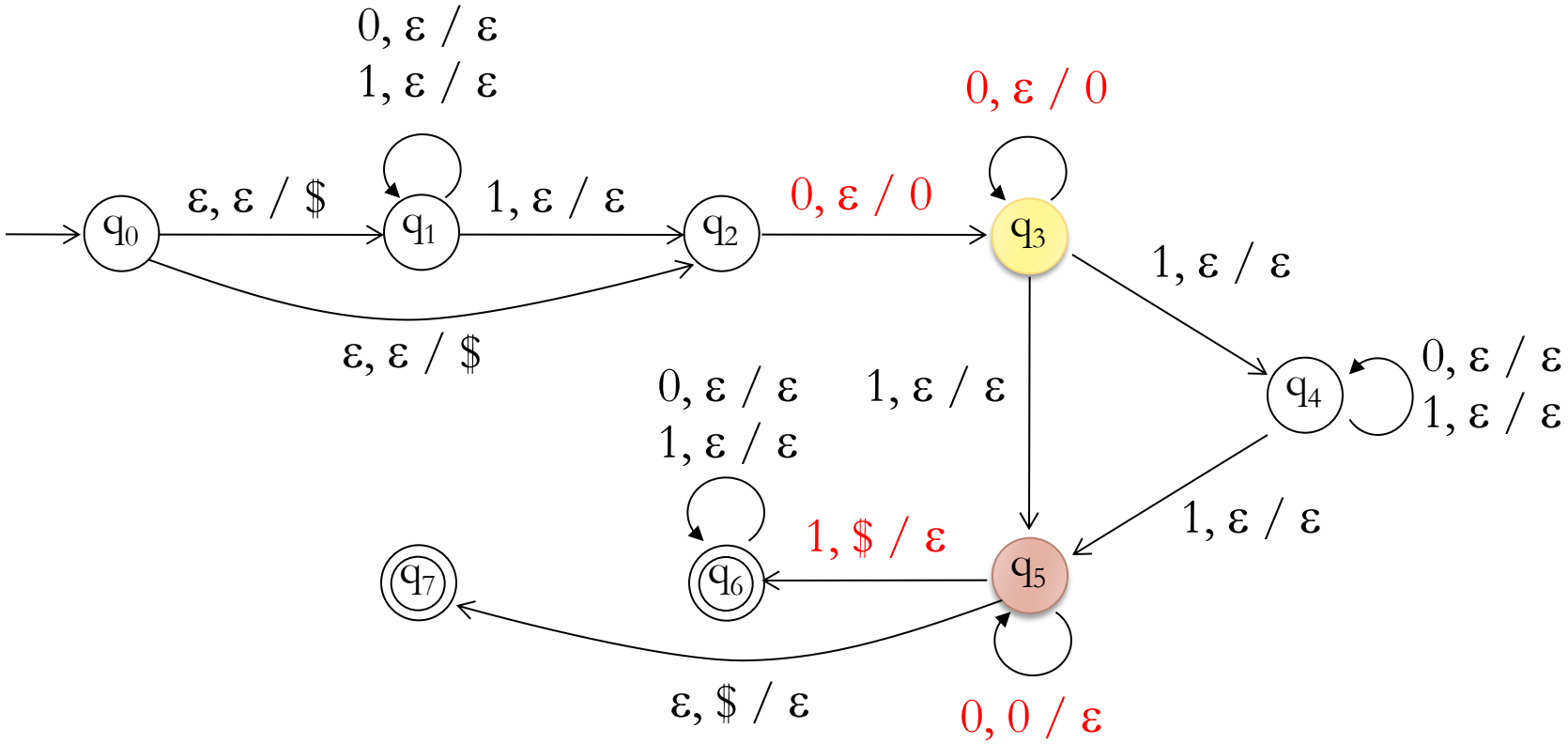
Guess where second block starts

Match 0s from stack



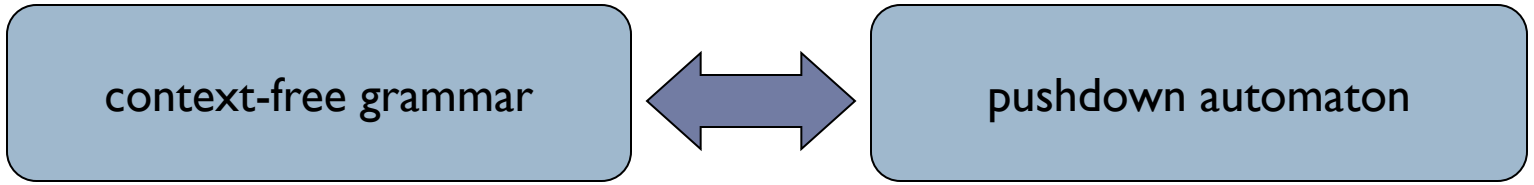
Example 4

$L = \{w: w \text{ has two 0-blocks with same number of 0s}\}$



Main theorem

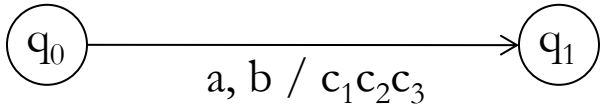
A language L is context-free if and only if it is accepted by some pushdown automaton



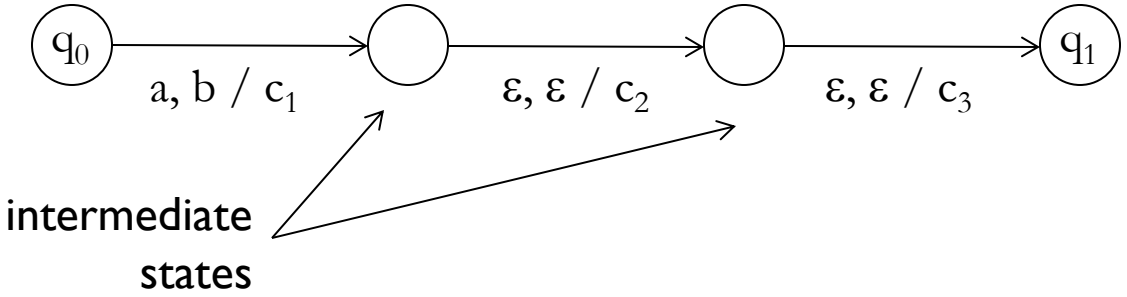


A convention

- Sometimes we denote "transitions" by:



- This will mean:



pop b, then push c_1, c_2 and c_3



From CFGs to PDAs

- Idea: Use PDA to simulate derivations

CFG:

$$\begin{aligned} A &\to 0A1 \\ A &\to B \\ B &\to \# \end{aligned}$$

$$A \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 00B11 \Rightarrow 00\#11$$

PDA control:	stack:	input:
write start variable	$A	00#11
replace production in reverse	$1A0	00#11
pop terminals and match	$1A	0#11
replace production in reverse	$11A0	0#11
pop terminals and match	$11A	#11
replace production in reverse	$11B	#11



From CFGs to PDAs

- If, after reading whole input, PDA ends up with an empty stack, derivation must be valid

- Conversely, if there is no valid derivation, PDA will get stuck somewhere
  - Either unable to match next input symbol,
  - Or match whole input but stack non empty

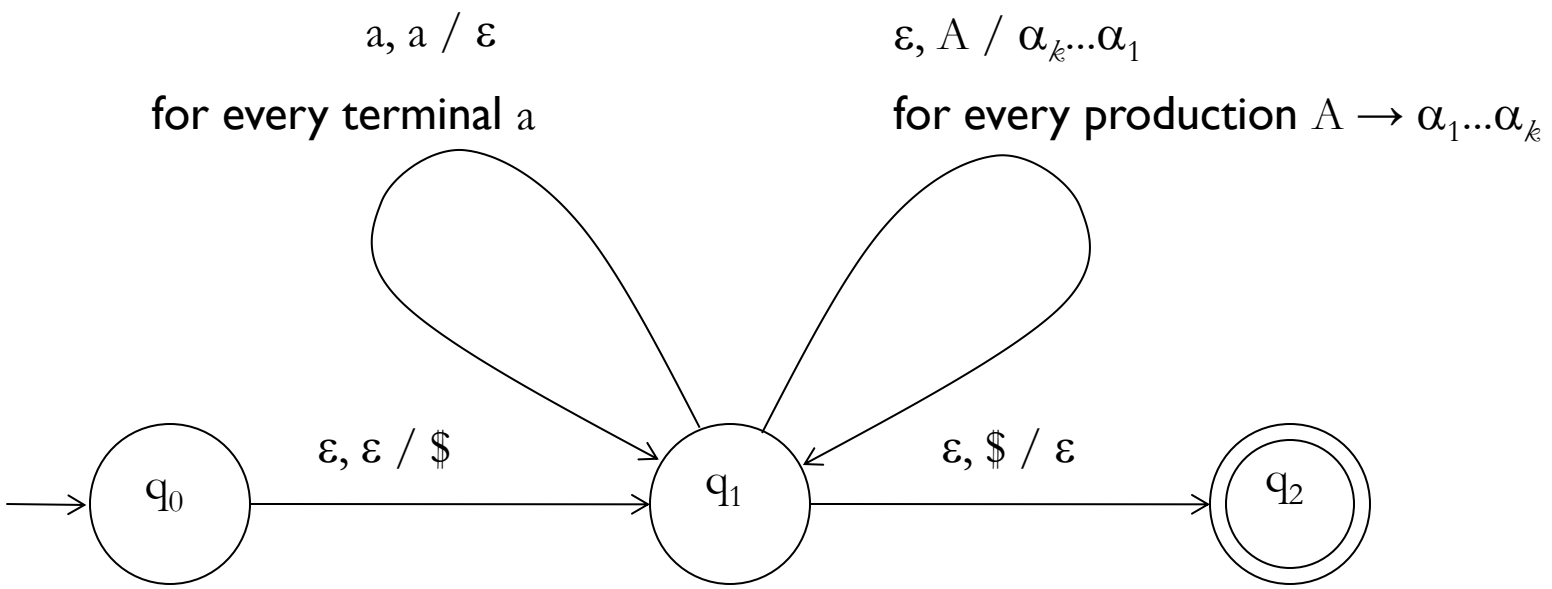


Description of PDA for CFG

- Push $ onto stack
- Repeat the following steps:
  - If the top of the stack is a variable A:
    Choose a rule $A \rightarrow \alpha$ and substitute A with α
  - If the top of the stack is a terminal a:
    Read next input symbol and compare to a
    If they don't match, reject
  - If top of stack is $, go to accept state

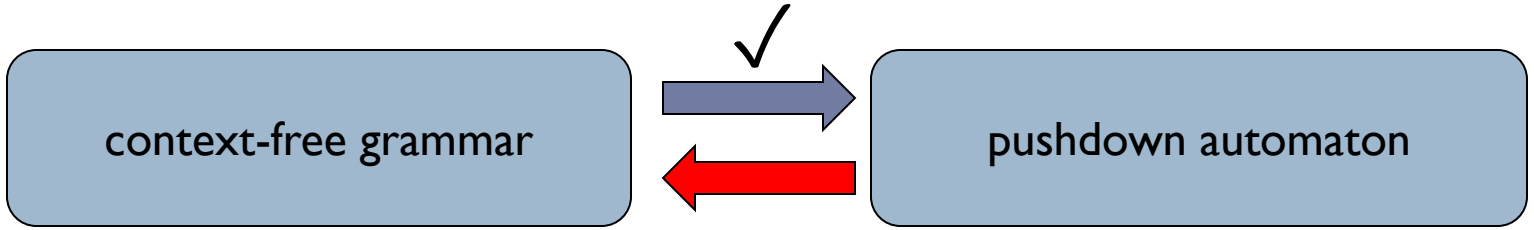


Description of PDA for CFG





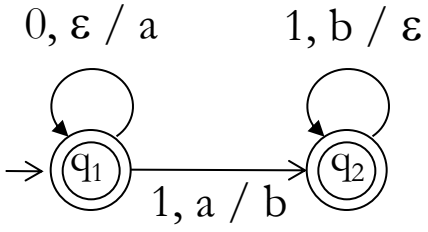
From PDAs to CFGs



- First, we simplify the PDA:
  - It has a single accept state q_f
  - It empties its stack before accepting
  - Each transition is either a push, or a pop, but not both



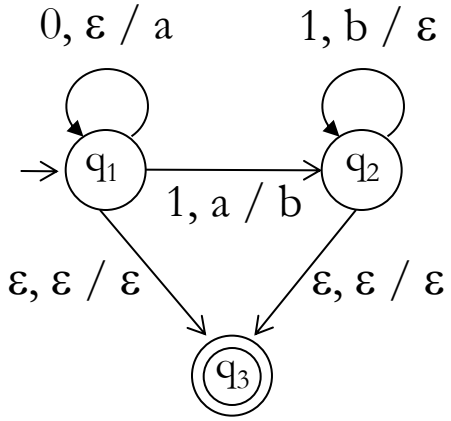
Simplifying the PDA



- First, we simplify the PDA:
  - It has a single accept state q_f
  - It empties its stack before accepting
  - Each transition is either a push, or a pop, but not both



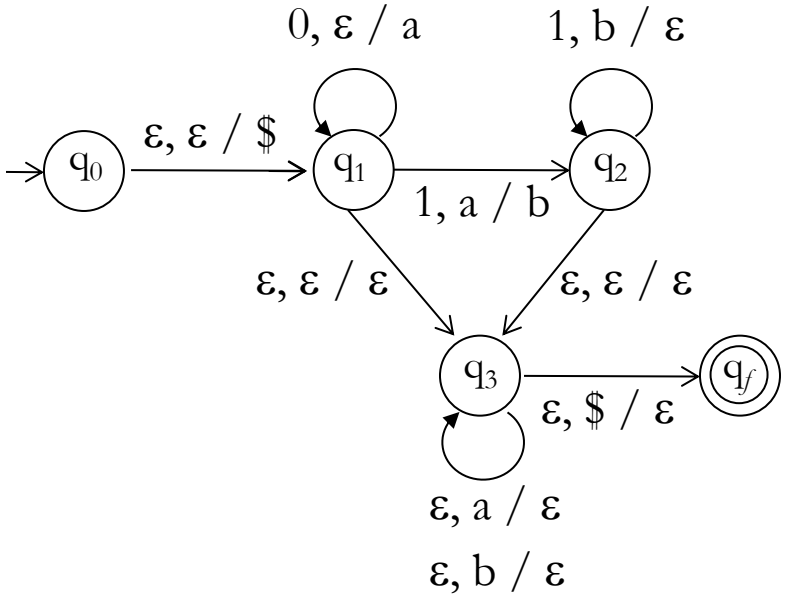
Simplifying the PDA



- First, we simplify the PDA:
  - It has a single accept state q_f
  - It empties its stack before accepting
  - Each transition is either a push, or a pop, but not both



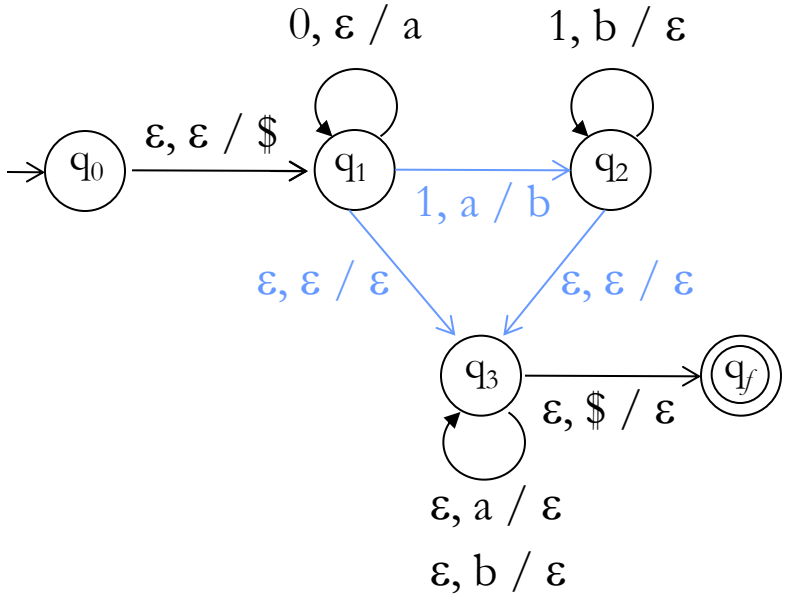
Simplifying the PDA



- First, we simplify the PDA:
  - It has a single accept state q_f
  - It empties its stack before accepting
  - Each transition is either a push, or a pop, but not both



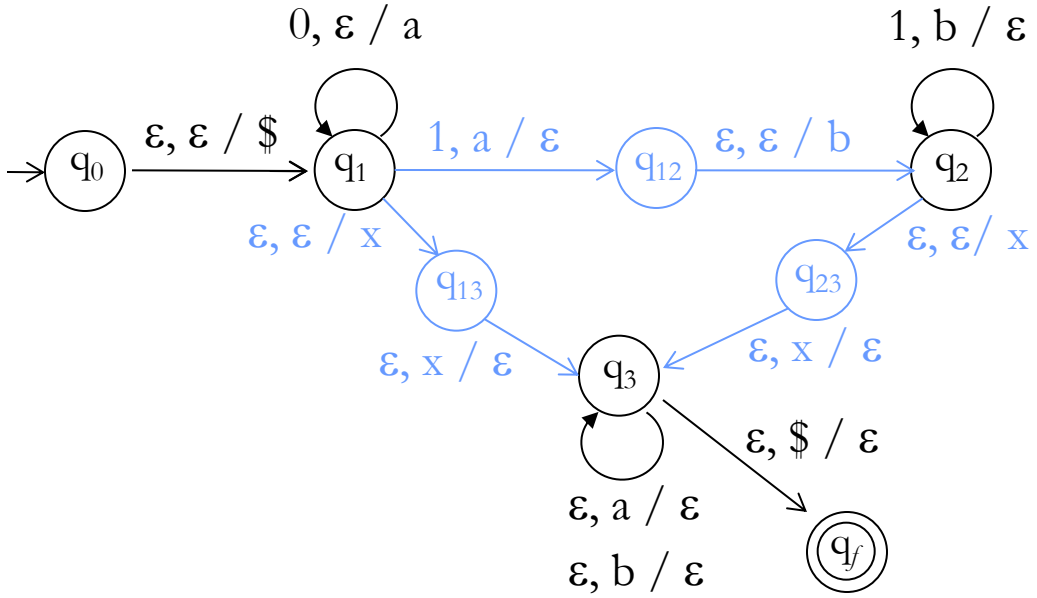
Simplifying the PDA



- First, we simplify the PDA:
  - It has a single accept state q_f
  - It empties its stack before accepting
  - Each transition is either a push, or a pop, but not both



Simplifying the PDA



- First, we simplify the PDA:
  - It has a single accept state q_f
  - It empties its stack before accepting
  - Each transition is either a push, or a pop, but not both



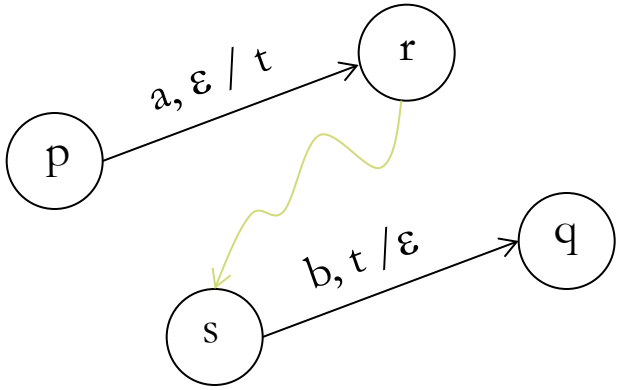
From PDAs to CFGs

- Let P=(Q, Σ, Γ, δ, q_0, F) be the PDA
- Construct the CFG G=(V,Σ',R,S),
  - where V={A_{pq} | for all p,q∈Q}
  - A_{pq} = {*x*: *x* leads from state p with an empty stack to state q with an empty stack}

- So when reading A_{pq} from p, we end with state q and preserve the stack



From PDAs to CFGs

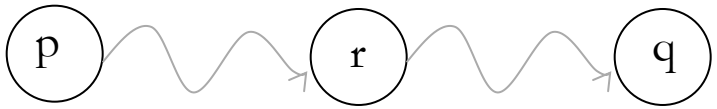


variables: A_{pq}

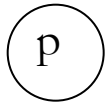
start variable: A_{0f}

(when push and pop the same symbol)

$$A_{pq} \rightarrow aA_{rs}b$$



$$A_{pq} \rightarrow A_{pr}A_{rq}$$

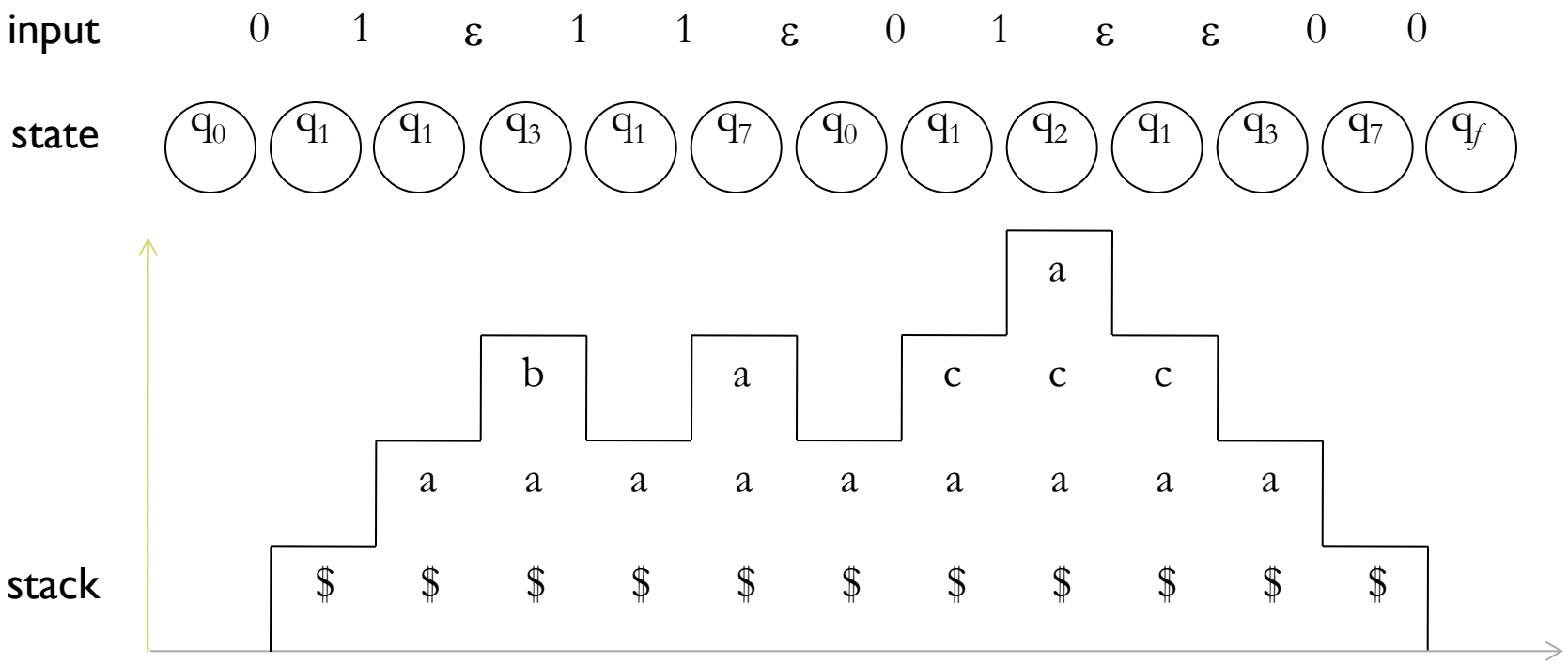


$$A_{pp} \rightarrow \varepsilon$$



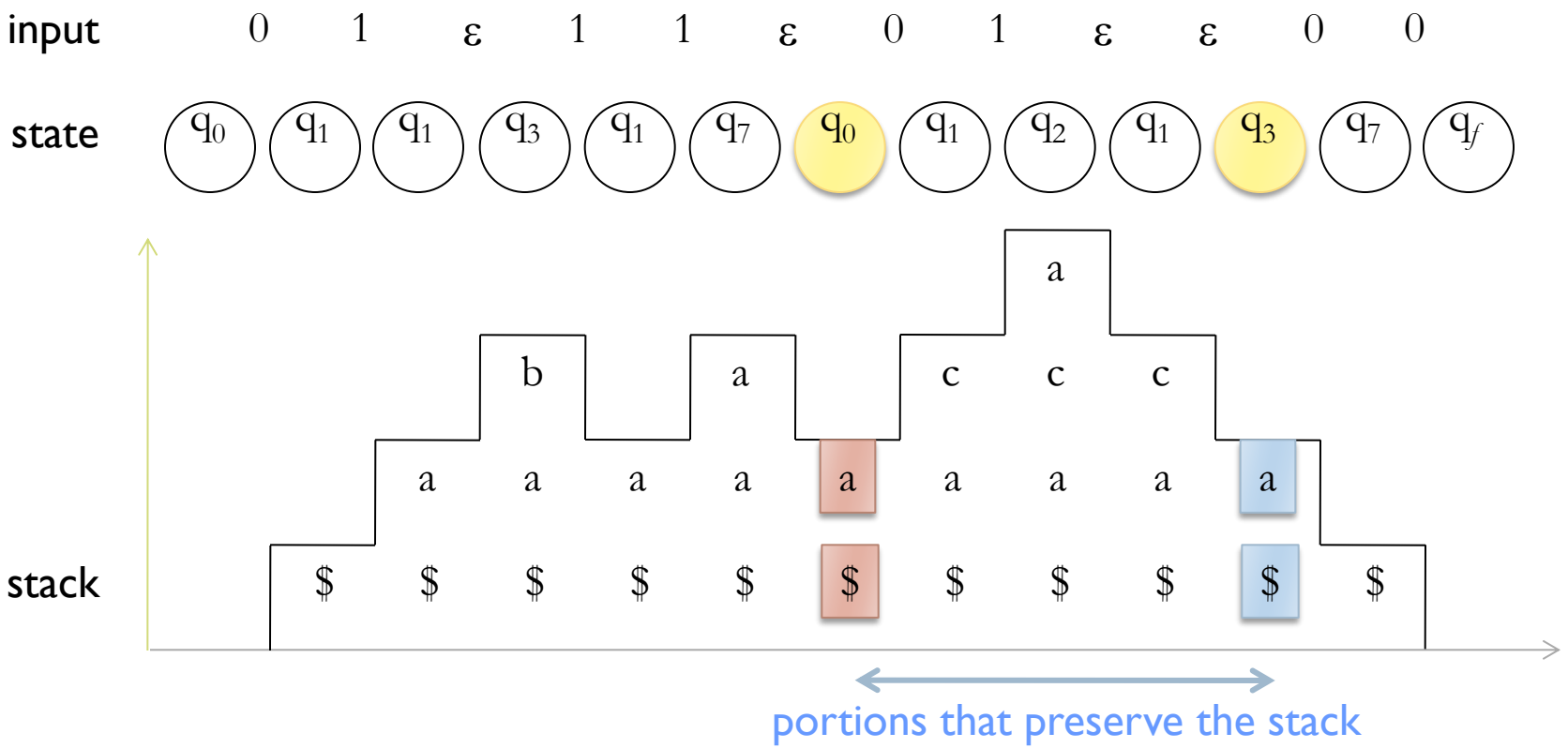
From PDAs to CFGs

- We look at the stack in an accepting computation:



From PDAs to CFGs

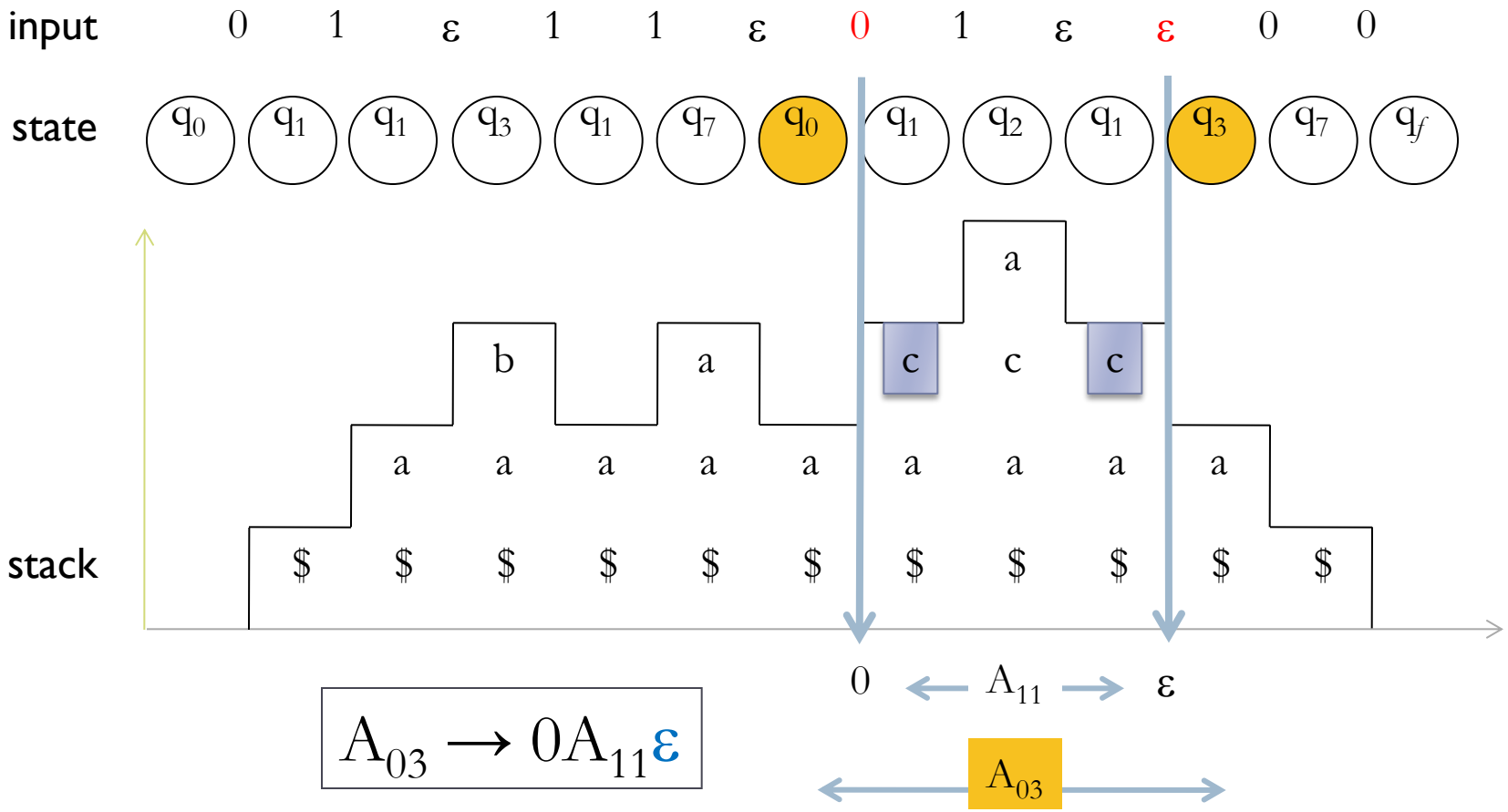
- We look at the stack in an accepting computation:



$A_{03} = \{x: x \text{ leads from } q_0 \text{ to } q_3 \text{ and preserves the stack}\}$

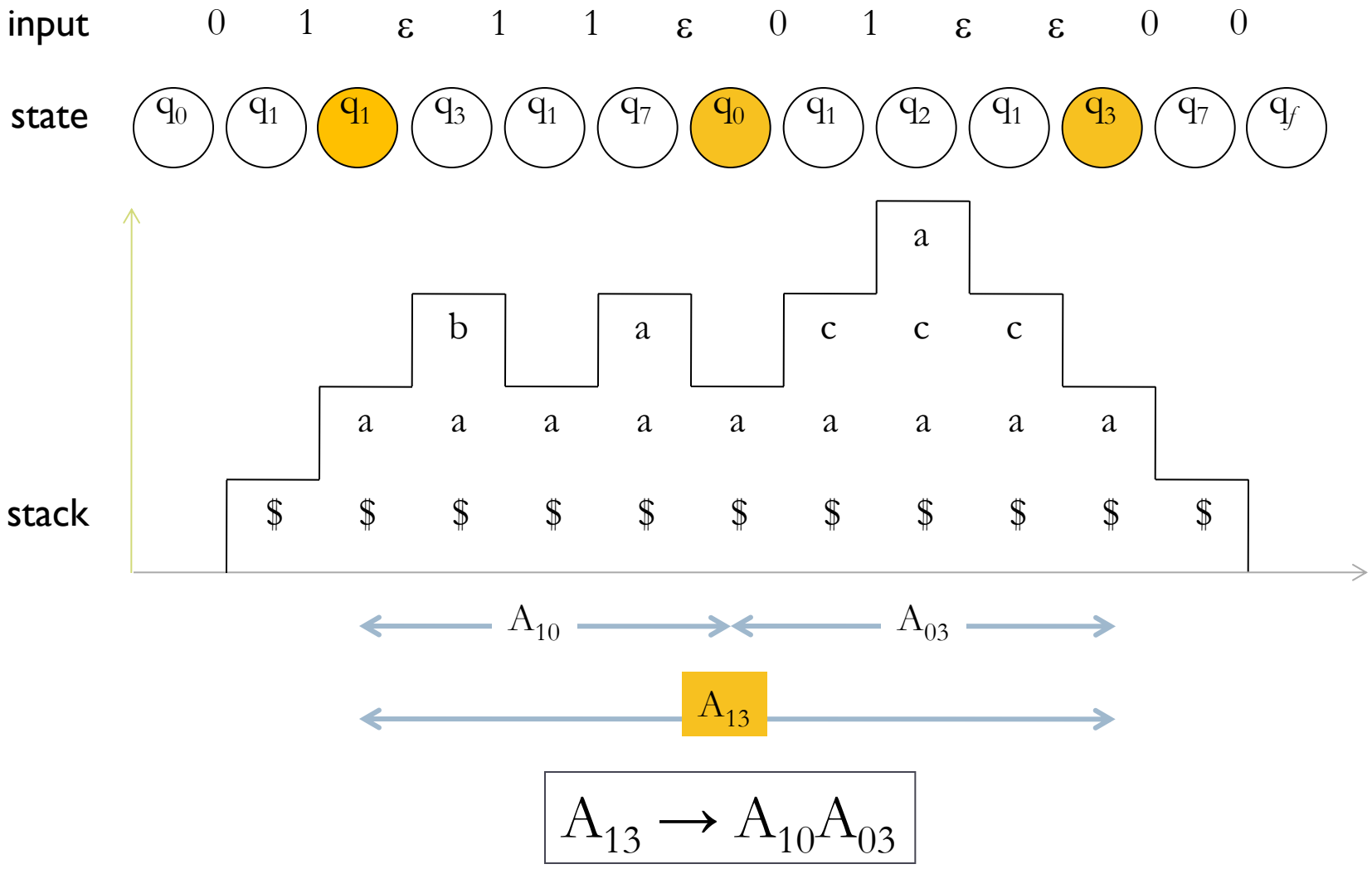


From PDAs to CFGs

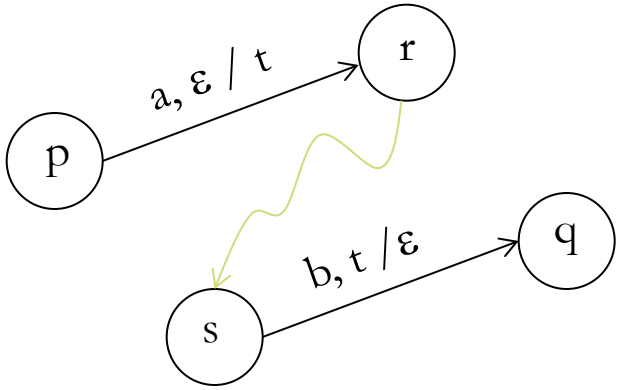


Since q_0 reads 0 and pushes a "c" *(and goes to q_1)*, q_1 reads ε and pops "c"

From PDAs to CFGs



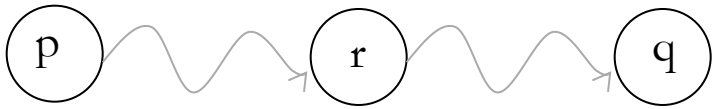
From PDAs to CFGs



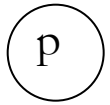
variables: A_{pq}

start variable: A_{0f}

$A_{pq} \rightarrow aA_{rs}b$



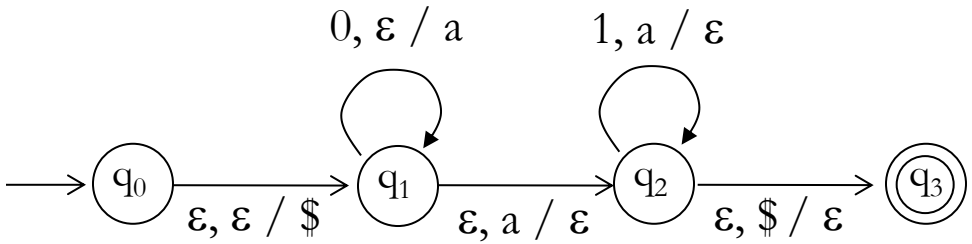
$A_{pq} \rightarrow A_{pr}A_{rq}$



$A_{pp} \rightarrow \varepsilon$

We can prove this CFG is equivalent to the PDA by induction, for details see Sipser's book

Example



start variable: A_{03}

productions:

$A_{00} \rightarrow A_{00}A_{00}$
$A_{00} \rightarrow A_{01}A_{10}$
$A_{00} \rightarrow A_{02}A_{20}$
$A_{00} \rightarrow A_{03}A_{30}$
$A_{01} \rightarrow A_{01}A_{11}$
$A_{01} \rightarrow A_{02}A_{21}$
...

$A_{00} \rightarrow \varepsilon$
$A_{11} \rightarrow \varepsilon$
$A_{22} \rightarrow \varepsilon$
$A_{33} \rightarrow \varepsilon$

$A_{12} \rightarrow 0A_{12}1$
$A_{12} \rightarrow 0A_{11}$

$A_{03} \rightarrow A_{12}$



PDA

- The definition of PDA is non-deterministic! So it's not a realistic machine!
  - PDA is only a theoretical counterpart of CFG



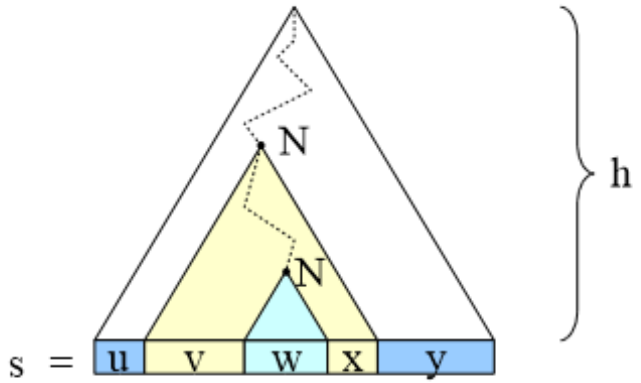
Pumping Lemma for CFL

Pumping Lemma for CFL

- If A is a context-free language, then there is a number p (the pumping length) where, if s is any string in A of length at least p, then s may be divided into 5 pieces s=uvxyz, satisfying:
  - For each $i \geq 0$, $uv^ixy^iz \in A$
  - $|vy| > 0$
  - $|vxy| \leq p$

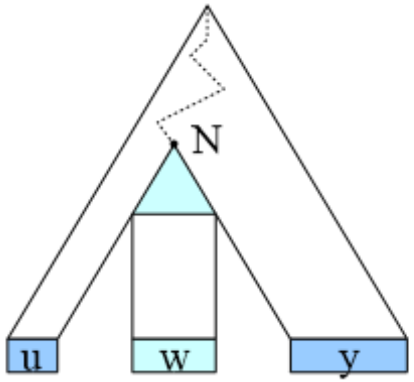


Proof – how large is the pumping length p?

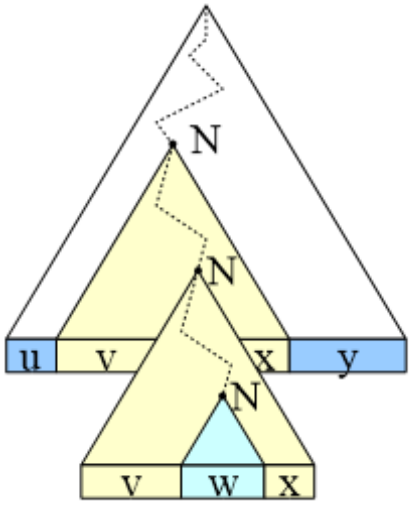


Sufficiently high derivation tree

There must be a variable N appearing twice



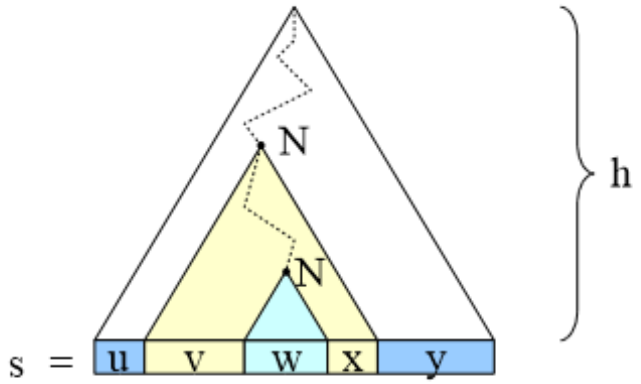
Generates uwy



Generates uv^iwx^iy



Proof – how large is the pumping length p?



Sufficiently high derivation tree

There must be a variable N appearing twice

Let v be the number of variables, b be the maximum number of symbols in the right-hand side of the rules

The height h needs to be v+1
so $p=b^{v+1}$

Example

- Show that the language $\{a^n b^n c^n \mid n \geq 0\}$ is not context-free



Example

- Show that the language $\{a^n b^n c^n \mid n \geq 0\}$ is not context-free.

  - Consider s=uvxyz
  - If v and y both contain only one type of symbols, then we cannot make the numbers of a,b,c equal
  - If either v or y contain more than one type of symbols, then $uv^i xy^i z$ won't contain them in correct order

  - *You can see CFL is not closed under intersection?*



Example

- Show that the language $\{ww \mid w \in \{0,1\}^*\}$ is not context-free



Example

- Show that the language {ww | w∈{0,1}*} is not context-free
  - Consider $s=0^p1^p0^p1^p$, where p is the pumping length
  - in s=uvxyz, vxy must "straddle" 0 and 1, which must violate ww



Example

- Show that the language $\{a^i b^j c^k \mid 0 \leq i \leq j \leq k\}$ is not context-free



Example

- Show that the language $\{a^i b^j c^k \mid 0 \leq i \leq j \leq k\}$ is not context-free

  - consider $s = a^P b^P c^P$
  - we can "pump down" as well as "pump up"



Chomsky Normal Form and Parsing Algorithms

Chomsky Normal Form for CFG

- One of the simplest form of CFG

- A context-free grammar is in *Chomsky normal form* if every rule is of the form

$$A \rightarrow BC$$

$$A \rightarrow a$$

  - where a is any terminal and A,B,C are any variables
  - except that B and C may not be the start variable
  - we permit the rule $S \rightarrow \varepsilon$, where S is the start variable

Since there is no rule $A \rightarrow \varepsilon$ for non-start variable A, ε can only be generated by the $S \rightarrow \varepsilon$ rule



Remove the following forms

- Start variable S at the right-hand side of a rule
- ε rules: $A \rightarrow \varepsilon$ for non-start variable A
- Unit rules: $A \rightarrow B$
- Rules with ≥3 symbols and rules with one terminal and another symbol on the right-hand side

- Then we can change into Chomsky normal form easily



Transform into Chomsky normal form

- Start variable S at the right-hand side of a rule
  - Add a new start variable $S_0 \rightarrow S$
- ε rules: $A \rightarrow \varepsilon$ for non-start variable A
- Unit rules: $A \rightarrow B$
- Rules with ≥3 symbols and rules with one terminal and another symbol on the right-hand side



Transform into Chomsky normal form

- Start variable S at the right-hand side of a rule
  - Add a new start variable $S_0 \rightarrow S$
- ε rules: $A \rightarrow \varepsilon$ for non-start variable A
  - For each occurrence of A on the right-hand side of a rule, we add a new rule with the occurrence deleted

    e.g. $R \rightarrow uAvAw$

    add: $R \rightarrow uvAw$ $R \rightarrow uAvw$ $R \rightarrow uvw$
  - if we have rule $R \rightarrow A$, repeat with R unless we had previously removed with the rule $R \rightarrow \varepsilon$
- Unit rules: $A \rightarrow B$
- Rules with ≥3 symbols and rules with one terminal and another symbol on the right-hand side



Transform into Chomsky normal form

- Start variable S at the right-hand side of a rule
- ε rules: $A \rightarrow \varepsilon$ for non-start variable A
- Unit rules: $A \rightarrow B$
  - For every rule $B \rightarrow u$ (u is any string of variables and terminals)
  - add the rule $A \rightarrow u$, unless this was a unit rule previously removed
- Rules with ≥3 symbols and rules with one terminal and another symbol on the right-hand side
  - For $A \rightarrow u_1 u_2 \ldots u_k$, where $k \geq 3$ and each u_i is a variable or terminal
  - Replace with the rules:

    $A \rightarrow u_1 A_1$, $A_1 \rightarrow u_2 A_2$, $A_2 \rightarrow u_3 A_3$, …, $A_{k-2} \rightarrow u_{k-1} u_k$
- Finally replace any terminal u in the rule $A \rightarrow uv$ with a new variable U: $U \rightarrow u$



Example

- The CFG:

$$S \rightarrow ASA \mid aB$$
$$A \rightarrow B \mid S$$
$$B \rightarrow b \mid \varepsilon$$

- add S_0:

$$S_0 \rightarrow S$$
$$S \rightarrow ASA \mid aB$$
$$A \rightarrow B \mid S$$
$$B \rightarrow b \mid \varepsilon$$

- remove $B \rightarrow \varepsilon$:

$$S_0 \rightarrow S$$
$$S \rightarrow ASA \mid aB \mid a$$
$$A \rightarrow B \mid S \mid \varepsilon$$
$$B \rightarrow b$$

- further remove $A \rightarrow \varepsilon$:

$$S_0 \rightarrow S$$
$$S \rightarrow ASA \mid aB \mid a \mid SA \mid AS \mid S$$
$$A \rightarrow B \mid S$$
$$B \rightarrow b$$



Example

- The CFG:

$S_0 \rightarrow S$
$S \rightarrow ASA \mid aB \mid a \mid SA \mid AS \mid S$
$A \rightarrow B \mid S$
$B \rightarrow b$

- remove $S \rightarrow S$:

$S_0 \rightarrow S$
$S \rightarrow ASA \mid aB \mid a \mid SA \mid AS$
$A \rightarrow B \mid S$
$B \rightarrow b$

- further remove $S_0 \rightarrow S$:

$S_0 \rightarrow ASA \mid aB \mid a \mid SA \mid AS$
$S \rightarrow ASA \mid aB \mid a \mid SA \mid AS$
$A \rightarrow B \mid S$
$B \rightarrow b$



Example

- The CFG:

$S_0 \rightarrow ASA \mid aB \mid a \mid SA \mid AS$
$S \rightarrow ASA \mid aB \mid a \mid SA \mid AS$
$A \rightarrow B \mid S$
$B \rightarrow b$

- remove A→B:

$S_0 \rightarrow ASA \mid aB \mid a \mid SA \mid AS$
$S \rightarrow ASA \mid aB \mid a \mid SA \mid AS$
$A \rightarrow S \mid b$
$B \rightarrow b$

- further remove A→S:

$S_0 \rightarrow ASA \mid aB \mid a \mid SA \mid AS$
$S \rightarrow ASA \mid aB \mid a \mid SA \mid AS$
$A \rightarrow b \mid ASA \mid aB \mid a \mid SA \mid AS$
$B \rightarrow b$



Example

- Solve with:

$S_0 \rightarrow ASA \mid aB \mid a \mid SA \mid AS$
$S \rightarrow ASA \mid aB \mid a \mid SA \mid AS$
$A \rightarrow b \mid ASA \mid aB \mid a \mid SA \mid AS$
$B \rightarrow b$

- Finally:

$S_0 \rightarrow AA_1 \mid UB \mid a \mid SA \mid AS$
$S \rightarrow AA_1 \mid UB \mid a \mid SA \mid AS$
$A \rightarrow b \mid AA_1 \mid UB \mid a \mid SA \mid AS$
$B \rightarrow b$
$A_1 \rightarrow SA$
$U \rightarrow a$



Cocke–Younger–Kasami algorithm

- Pushdown Automaton is not a real algorithm
- Parsing algorithm for Chomsky normal form of CFG
- simple dynamic programming

- Given a CFG G in CNF and a string `s[1…n]`, check whether `s` is in G



Cocke–Younger–Kasami algorithm

- Given a CFG G in CNF and a string s[1…n], check whether s is in G
  - G has r variables $R_1, \ldots, R_r$ with start variable R_1
- Matrix T[1…n+1, 1…n+1]:
  - each element is a subset of {1, …, r}
  - k∈T[i, j]: string s[i…j−1] can be derived from variable R_k

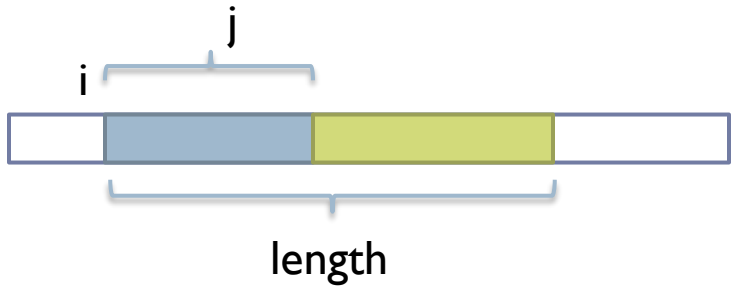
```
for i=1 to n
    for each unit rule Rk → s[i]
        add k into T[i,i+1];

for length=2 to n
    for i=1 to n-length+1
        for j=1 to length-1
            for each production Ra→RbRc
                if (b ∈ T[i,i+j] & c ∈ T[i+j,i+length])
                    add a into T[i,i+length];
```



Cocke–Younger–Kasami algorithm

- Given a CFG G in CNF and a string $s[1...n]$, check whether s is in G
  - G has r variables $R_1, ..., R_r$ with start variable R_1
- Matrix $T[1...n+1, 1...n+1]$:
  - each element is a subset of $\{1, ..., r\}$
  - $k \in T[i,j]$: string $s[i...j-1]$ can be derived from variable R_k




```
for i=1 to n
    for each unit rule Rk → s[i]
        add k into T[i,i+1];

for length=2 to n
    for i=1 to n-length+1
        for j=1 to length-1
            for each production Ra→RbRc
                if (b ∈ T[i,i+j] & c ∈ T[i+j,i+length])
                    add a into T[i,i+length];
```

If $1 \in T[1,n+1]$ then s is in G

Cocke–Younger–Kasami algorithm

- Time: $O(n^3|G|)$
  - can be improved to $O(n^\omega)$ if $|G|$ is a constant *[Valiant 1974]*

```
for i=1 to n
    for each unit rule R_k → s[i]
        add k into T[i,i+1];

for length=2 to n
    for i=1 to n-length+1
        for j=1 to length-1
            for each production R_a→R_bR_c
                if (b ∈ T[i,i+j] & c ∈ T[i+j,i+length])
                    add a into T[i,i+length];
```



Improvement

- Time: $O(n^3|G|)$
  - can be improved to $O(n^\omega)$ if $|G|$ is a constant *[Valiant 1974]*

- What about $T \Diamond T$:
  - looks more like a matrix multiplication
  - *time complexity for matrix multiplication $O(n^\omega)$, where $\omega < 2.371339$*

[Alman, Duan, Virginia V. Williams, Xu, Xu, Zhou 2025]

```
for i=1 to n+1
    for j=1 to n+1
        for k=1 to n+1
            for each production Ra→RbRc
                if (b ∈ T[i,k] & c ∈ T[k,j])
                    add a into W[i,j];
```



Improvement

- Define $T'[1\ldots(n+1)r,\ 1\ldots n+1]$ and $T''[1\ldots n+1,\ 1\ldots(n+1)r]$
  - $T'[(i-1)r+k,\ j]=1$ if $k \in T[i,j]$
  - $T''[i,\ (j-1)r+k]=1$ if $k \in T[i,j]$
- By $T' \cdot T''$, we can compute $T \lozenge T$:

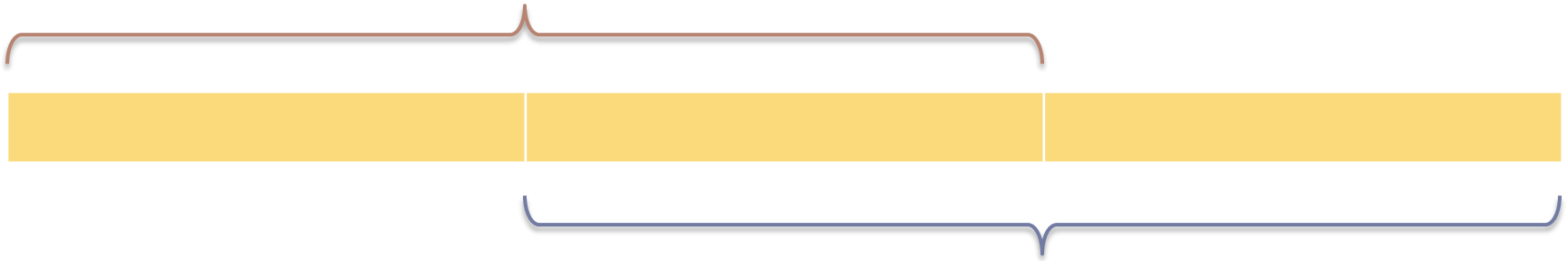
Time: $O(n^{\omega})$ if r is a constant

```
for i=1 to n+1
    for j=1 to n+1
        for k=1 to n+1
            for each production Ra→RbRc
                if (b ∈ T[i,k] & c ∈ T[k,j])
                    add a into W[i,j];
```





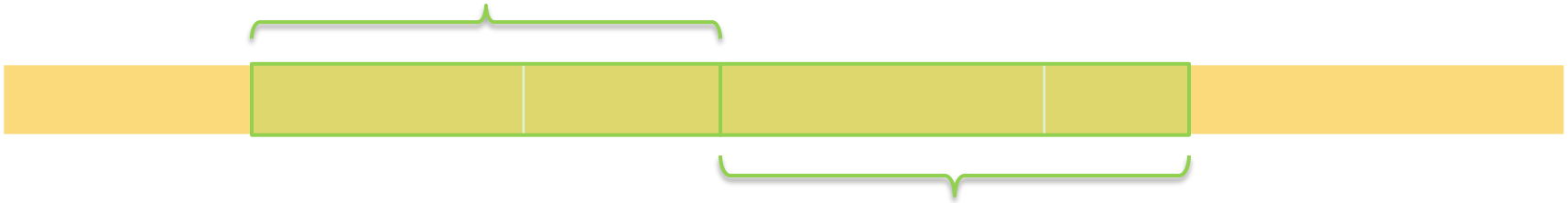
Improvement (1)



- We want to compute the $(n+1)\times(n+1)$ matrix for $s[1\ldots n]$ (based on current matrix)
  - Recursively solve $s[1\ldots 2n/3]$ and $s[n/3\ldots n]$ to get the matrices T_1 and T_2
  - $T=T_1 \cup T_2$
  - compute $W=T \lozenge T$
  - delete the middle part $s[n/3\ldots 2n/3]$, and recursively solve it based on W



Improvement (1)



- We want to compute the $(n+1)\times(n+1)$ matrix for $s[1\ldots n]$
  - Recursively solve $s[1\ldots 2n/3]$ and $s[n/3\ldots n]$ to get the matrices T_1 and T_2
  - $T=T_1 \cup T_2$
  - compute $W=T \lozenge T$
  - delete the middle part $s[n/3\ldots 2n/3]$, and recursively solve it based on W



Improvement (1)



- We want to compute the $(n+1)\times(n+1)$ matrix for $s[1\ldots n]$
  - Recursively solve $s[1\ldots 2n/3]$ and $s[n/3\ldots n]$ to get the matrices T_1 and T_2
  - $T=T_1 \cup T_2$
  - compute $W=T \lozenge T$
  - delete the middle part $s[n/3\ldots 2n/3]$, and recursively solve it based on W

$$T(n) = 3T\left(\frac{2}{3}n\right) + O(n^{\omega})$$

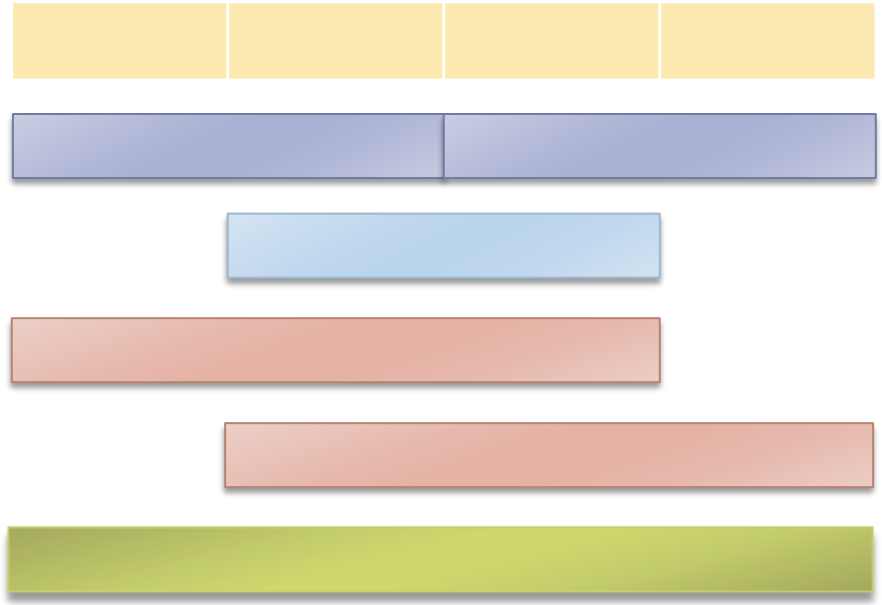
$$T(n) < n^{2.7096}$$



Improvement (2)

P2(s): s[1…n/2], s[n/2…n] are solved

1. Call P2(s[n/4…3n/4])
2. Call P3(s[1…3n/4]) and P3(s[n/4…n]) based on (1)
3. Call P4 based on (2)



P3(s): s[1…2n/3], s[n/3…n] are solved

1. Compute W=T∪(T◊T) for current matrix T
2. Delete s[n/3…2n/3], then call P2 based on W

P4(s): s[1…3n/4], s[n/4…n] are solved

1. Compute W=T∪(T◊T) for current matrix T
2. Delete s[n/4…3n/4], then call P2 based on W



Improvement (2)

P2(s): s[1…n/2], s[n/2…n] are solved

1. Call P2(s[n/4…3n/4])
2. Call P3(s[1…3n/4]) and P3(s[n/4…n]) based on (1)
3. Call P4 based on (2)

P3(s): s[1…2n/3], s[n/3…n] are solved

1. Compute W=T∪(T◊T) for current matrix T
2. Delete s[n/3…2n/3], then call P2 based on W

P4(s): s[1…3n/4], s[n/4…n] are solved

1. Compute W=T∪(T◊T) for current matrix T
2. Delete s[n/4…3n/4], then call P2 based on W

Main algorithm:

- Recursively solve s[1…n/2] and s[n/2…n]
- Call P2(s)

$$T(n) = 2T\left(\frac{n}{2}\right) + T_2(n) + O(n^2)$$

$$T_2(n) = T_2\left(\frac{n}{2}\right) + 2T_3\left(\frac{3n}{4}\right) + T_4(n)$$

$$T_3(n) = T_2\left(\frac{2n}{3}\right) + O(n^\omega)$$

$$T_4(n) = T_2\left(\frac{n}{2}\right) + O(n^\omega)$$

$$T_2(n) = 4T_2\left(\frac{n}{2}\right) + O(n^\omega)$$

$$T_2(n) = O(n^\omega)$$



Improvement *[Valiant 1974]*

P2(s): s[1…n/2], s[n/2…n] are solved

1. Call P2(s[n/4…3n/4])
2. Call P3(s[1…3n/4]) and P3(s[n/4…n]) based on (1)
3. Call P4 based on (2)

P3(s): s[1…2n/3], s[n/3…n] are solved

1. Compute W=T∪(T◊T) for current matrix T
2. Delete s[n/3…2n/3], then call P2 based on W

P4(s): s[1…3n/4], s[n/4…n] are solved

1. Compute W=T∪(T◊T) for current matrix T
2. Delete s[n/4…3n/4], then call P2 based on W

Main algorithm:

- Recursively solve s[1…n/2] and s[n/2…n]
- Call P2(s)

$$T(n) = 2T\left(\frac{n}{2}\right) + T_2(n) + O(n^2)$$

$$T(n) = O(n^{\omega})$$

